INTERVIEW PREPARATION SERIES

SQL Server Interview Playbook

The 50 most-asked SQL interview questions, answered for Microsoft SQL Server, with diagrams, T-SQL examples and the follow-ups interviewers ask next.

50	6	18	2026
questions	parts	diagrams	edition

Data with Deepankar

@datawithdeepankar

How to use this playbook

This playbook covers the 50 SQL questions that come up most often in interviews at service companies (TCS, Infosys, Wipro, Accenture, Cognizant), global capability centres (JP Morgan, Goldman Sachs, Wells Fargo, Walmart Labs, Target, AmEx) and Indian product companies (Razorpay, Swiggy, Cred, Meesho, Zerodha, Flipkart, PhonePe).

Every answer is written for **Microsoft SQL Server** and T-SQL. Where another database behaves differently, it is mentioned only so you can answer a comparison question confidently.

The questions are arranged as a learning path. Parts 1 to 3 build query-writing skill, Part 4 adds modern T-SQL features, and Parts 5 and 6 cover the engine-level topics that separate a fresher from an experienced developer.

EACH QUESTION FOLLOWS THE SAME LAYOUT

The interviewer asks
The question as you are likely to hear it.

Answer in 30 seconds
Say this first. It shows you know the core idea.

Build the answer step by step
The structure for a fuller two-minute answer.

Edge cases that impress
The details interviewers wait to hear.

Avoid saying
Common statements that cost candidates the round.

Likely follow-up
The next question, with a model answer.

A TWO-PASS STUDY PLAN

Pass 1 (about 2 hours): read only the question and the 30-second answer for all 50. Say each answer aloud.

Pass 2: for every question you could not explain comfortably, work through the steps, the diagram and the code. Run the code against the sample schema on the next page.

PRACTICE SCHEMA USED IN THE EXAMPLES

All examples use a simplified ShopSmart India retail database. Create these tables in a practice database to run every snippet.

dbo.Customers

```
CustomerID int PK
CustomerName, Email, Phone
City, State varchar
IsActive bit, LastSeen datetime2
```

dbo.Orders

```
OrderID int PK
CustomerID, SellerID, StoreID int
OrderDate date, Amount decimal(12,2)
Status varchar(20)
```

dbo.Payments

```
PaymentID int PK
OrderID int, PaidAmount decimal
Status varchar(20)
```

dbo.Employees

```
EmpID int PK, EmpName
ManagerID int NULL, DeptID int
Salary decimal(12,2)
```

dbo.Wallets

```
WalletID int PK, CustomerID int
Balance decimal(12,2)
```

dbo.Products / Sellers / Stores

```
ProductID, ProductName, Attributes
nvarchar(max) JSON
SellerID, BusinessName · StoreID, StoreName
```

Contents

Part 1 · Join Foundations & NULL Logic (Q01–Q06)			**5**
Q01	INNER vs LEFT vs RIGHT vs FULL OUTER JOIN	Easy	6
Q02	Self-Join: The Employee-Manager Pattern	Easy	7
Q03	CROSS JOIN: Intentional vs Accidental	Easy	8
Q04	COALESCE, NULLIF, ISNULL and IS DISTINCT FROM	Easy	9
Q05	Subquery in SELECT vs FROM vs WHERE	Medium	10
Q06	Correlated Subquery: What and Why	Medium	11
Part 2 · Existence Checks & Advanced Joins (Q07–Q11)			**13**
Q07	Anti-Join and the NOT IN NULL Trap	Medium	14
Q08	EXISTS vs IN vs JOIN	Medium	15
Q09	Semi-Join	Medium	16
Q10	CROSS APPLY and OUTER APPLY	Hard	17
Q11	Does Join Order Matter?	Medium	18
Part 3 · Window Functions (Q12–Q22)			**19**
Q12	ROW_NUMBER vs RANK vs DENSE_RANK	Easy	20
Q13	PARTITION BY vs GROUP BY	Easy	21
Q14	Window Function vs Aggregate in the Same Query	Medium	22
Q15	Running Totals	Easy	23
Q16	LAG and LEAD	Medium	24
Q17	Frame Clauses: ROWS vs RANGE	Hard	25
Q18	The FIRST_VALUE / LAST_VALUE Frame Trap	Hard	26
Q19	NTILE	Medium	27
Q20	PERCENT_RANK and CUME_DIST	Hard	28
Q21	The Filter-Window Trap	Hard	29
Q22	Top-N per Group	Medium	30
Part 4 · Modern T-SQL Toolkit (Q23–Q30)			**32**
Q23	Recursive CTE	Medium	33
Q24	CTE vs Subquery vs #Temp Table vs @Table Variable	Medium	34
Q25	GROUPING SETS, ROLLUP and CUBE	Medium	35
Q26	Upsert with MERGE (and Its Safer Alternative)	Medium	36
Q27	OUTPUT Clause and Updatable CTEs in DML	Medium	37
Q28	JSON in SQL Server	Medium	39
Q29	Indexed Views (SQL Server's Materialized Views)	Medium	40
Q30	Table Partitioning	Hard	41
Part 5 · Indexes & Query Performance (Q31–Q40)			**43**
Q31	How a B-tree Index Works	Medium	44

Q32	Clustered vs Nonclustered Index	Hard	45
Q33	Composite Index Column Order	Medium	46
Q34	Covering Index and the INCLUDE Clause	Hard	47
Q35	Index Seek vs Index Scan vs Table Scan	Medium	48
Q36	Reading Execution Plans and Query Cost	Hard	50
Q37	Why Isn't My Index Being Used? (SARGability)	Medium	51
Q38	Low-Cardinality Columns and Filtered Indexes	Hard	52
Q39	Hash Index vs B-tree in SQL Server	Hard	53
Q40	The Cost of Indexes: Maintenance and Write Overhead	Medium	54
Part 6 · Transactions, Isolation & Concurrency (Q41–Q50)			**56**
Q41	ACID Explained Practically	Easy	57
Q42	Dirty Read vs Non-Repeatable Read vs Phantom Read	Medium	58
Q43	READ UNCOMMITTED and NOLOCK	Easy	59
Q44	READ COMMITTED (the Default)	Medium	60
Q45	REPEATABLE READ	Medium	61
Q46	SERIALIZABLE	Hard	62
Q47	Row Versioning: RCSI and SNAPSHOT Isolation	Hard	63
Q48	Optimistic vs Pessimistic Locking	Medium	65
Q49	Deadlocks: Detection and Retry	Medium	66
Q50	Lock Escalation	Hard	67

01

Join Foundations & NULL Logic

How rows from two tables meet, what happens when they don't, and why NULL quietly changes the answer.

Q01	INNER vs LEFT vs RIGHT vs FULL OUTER JOIN	Easy
Q02	Self-Join: The Employee-Manager Pattern	Easy
Q03	CROSS JOIN: Intentional vs Accidental	Easy
Q04	COALESCE, NULLIF, ISNULL and IS DISTINCT FROM	Easy
Q05	Subquery in SELECT vs FROM vs WHERE	Medium
Q06	Correlated Subquery: What and Why	Medium

01

PART 1 · JOIN FOUNDATIONS & NULL LOGIC

INNER vs LEFT vs RIGHT vs FULL OUTER JOIN

Easy | Joins | Asked at: TCS, Infosys, Accenture, Wipro

THE INTERVIEWER ASKS

> Explain the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN and FULL OUTER JOIN. Give a real example where each one returns a different set of rows.

ANSWER IN 30 SECONDS

INNER JOIN returns only rows that match on both sides. LEFT JOIN returns every row from the left table, with NULLs in the right-side columns where no match exists. RIGHT JOIN is the mirror image. FULL OUTER JOIN returns everything from both sides and fills NULLs wherever either side has no partner.

Orders

OrderID
1
2
3

Payments

OrderID
1
3
9

Order 2 has no payment. Payment 9 has no order.

INNER JOIN

o.ID	p.ID
1	1
3	3

LEFT JOIN

o.ID	p.ID
1	1
2	NULL
3	3

RIGHT JOIN

o.ID	p.ID
1	1
3	3
NULL	9

FULL OUTER JOIN

o.ID	p.ID
1	1
2	NULL
3	3
NULL	9

Figure 1. Which rows survive each join type (gold rows are the unmatched ones)

BUILD THE ANSWER STEP BY STEP

1. Frame the answer around **matched vs unmatched rows**. That is the concept being tested, not the syntax.
2. Use one concrete pair of tables: `dbo.Orders` and `dbo.Payments`. INNER gives orders that have a payment. LEFT gives all orders, including unpaid ones, which is how you find payment failures. RIGHT gives all payments, including orphan payments with no order. FULL OUTER gives both kinds of orphans in one result.
3. Point out that RIGHT JOIN is rare in production code. Teams swap the table order and write LEFT JOIN because it reads top-down.
4. Be ready to write the **anti-join**: `LEFT JOIN ... WHERE p.OrderID IS NULL` returns orders with no payment.

EDGE CASES THAT IMPRESS

- If one order has three payment rows, both INNER and LEFT JOIN repeat that order three times. Juniors forget that joins multiply rows.
- SQL Server fully supports `FULL OUTER JOIN`, unlike MySQL. No UNION workaround is needed.
- NULL never equals NULL, so rows with a NULL join key never match, not even to another NULL.

AVOID SAYING

- "LEFT JOIN returns all rows from both tables." It returns all rows from the **left** table only.
- Drawing only Venn circles. Interviewers want row-level reasoning: which rows survive and why.

Likely follow-up: You wrote a LEFT JOIN but put a condition on the right table in the WHERE clause. What happens?

The unmatched rows have NULL in that column, the WHERE condition evaluates to UNKNOWN, and they are filtered out. The LEFT JOIN silently behaves like an INNER JOIN. Move the right-table condition into the ON clause.

PRO TIP

Think of a wedding guest list (Orders) and the RSVP replies (Payments). INNER JOIN is guests who replied. LEFT JOIN is every guest, with a blank against those who never replied. The blanks are usually the rows the business actually cares about.

Orders that never received a payment T-SQL

```
SELECT o.OrderID, o.CustomerID, o.Amount
FROM dbo.Orders AS o
LEFT JOIN dbo.Payments AS p
       ON p.OrderID = o.OrderID
WHERE p.OrderID IS NULL;
```

02

PART 1 · JOIN FOUNDATIONS & NULL LOGIC

Self-Join: The Employee-Manager Pattern

Easy | Joins | Asked at: Infosys, Cognizant, TCS, Wells Fargo

THE INTERVIEWER ASKS

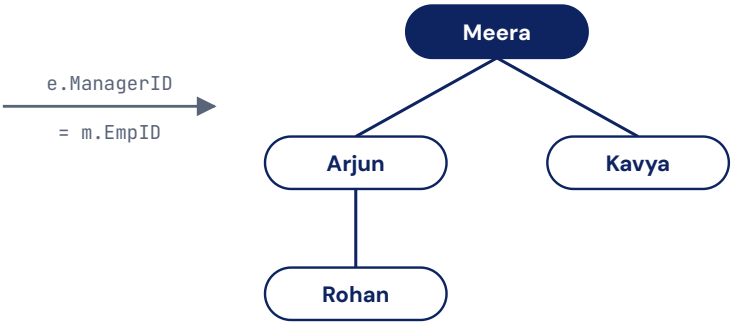
The `dbo.Employees` table has EmpID, EmpName and ManagerID, where ManagerID refers back to EmpID in the same table. Write a query that lists every employee with their manager's name.

ANSWER IN 30 SECONDS

Join the table to itself using two aliases, one for the employee row and one for the manager row, on `e.ManagerID = m.EmpID`. Use LEFT JOIN so the top-level person, who has no manager, is not dropped.

dbo.Employees (one table, two aliases)

EmpID	EmpName	ManagerID
1	Meera	NULL
2	Arjun	1
3	Kavya	1
4	Rohan	2



LEFT JOIN keeps Meera, whose ManagerID is NULL

Figure 2. A self-join reads the same table twice: once as employee, once as manager

BUILD THE ANSWER STEP BY STEP

1. A self-join is an ordinary join where both sides happen to be the same table. The aliases are what make them two logical tables.
2. Choose LEFT JOIN on purpose. The CEO's ManagerID is NULL, so an INNER JOIN removes the CEO from the result.

3. Mention related variations: employees who share the same manager, or employees who earn more than their manager.
4. If the interviewer asks for every level of the hierarchy, that is a recursive CTE (Question 23), not a single self-join.

EDGE CASES THAT IMPRESS

- Bad data can create a cycle (A manages B and B manages A). A single self-join is fine, but recursive queries need a depth guard.
- Use `ISNULL(m.EmpName, 'No manager')` or `COALESCE` so the CEO row displays cleanly.
- In a matrix organisation an employee may have two managers, usually stored in a bridge table. One output row per manager is then correct.

AVOID SAYING

- Using the same alias on both sides. SQL Server raises error 1011: the correlation name is specified multiple times.
- Writing INNER JOIN and claiming the top manager still appears.

Likely follow-up: Now show each employee, their manager, and their manager's manager.

Chain a second LEFT JOIN to the table with a third alias (`LEFT JOIN dbo.Employees AS gm ON m.ManagerID = gm.EmpID`). For unlimited depth, switch to a recursive CTE.

Employee with manager name T-SQL

```
SELECT e.EmpName                          AS Employee,
       ISNULL(m.EmpName, 'No manager')      AS Manager
FROM dbo.Employees AS e
LEFT JOIN dbo.Employees AS m
       ON m.EmpID = e.ManagerID;
```

PART 1 · JOIN FOUNDATIONS & NULL LOGIC

03 CROSS JOIN: Intentional vs Accidental

Easy | Joins | Asked at: TCS, Cognizant, Accenture, Infosys

THE INTERVIEWER ASKS

> What is a CROSS JOIN, and when would you actually want one?

ANSWER IN 30 SECONDS

A CROSS JOIN returns the Cartesian product: every row of table A paired with every row of table B, so N x M rows. Intentional uses include generating every store-date or size-colour combination. The accidental version happens when a join condition is forgotten, and the result set explodes.

BUILD THE ANSWER STEP BY STEP

1. Define it precisely: no join condition, N x M rows.
2. Give the reporting use case. To show zero sales on days a store sold nothing, CROSS JOIN stores with a calendar, then LEFT JOIN the sales onto that grid.
3. Describe the accidental form: `FROM dbo.Orders, dbo.Payments` with no WHERE condition is an implicit cross join, a classic production incident.
4. Recommend the explicit `CROSS JOIN` keyword so the intent is documented in the code.

EDGE CASES THAT IMPRESS

- A CROSS JOIN followed by a WHERE equality condition behaves like an INNER JOIN, but the explicit INNER JOIN syntax is clearer.
- Growth is fast: 1,000 x 1,000 is one million rows; 100,000 x 100,000 is ten billion.
- SQL Server 2022 added `GENERATE_SERIES`, which makes the calendar side easy to build. On older versions, use a numbers table.

AVOID SAYING

- "CROSS JOIN is always a mistake." Calendar grids and combination tables depend on it.
- Using the old comma syntax in new code. It hides intent.

Likely follow-up: Produce one row for every store and every day in the last 30 days, showing zero where there were no sales.

Generate the 30 dates, CROSS JOIN them with the stores table, then LEFT JOIN the orders on both store and date, and wrap the SUM in ISNULL(..., 0). The code block shows this.

PRO TIP

A restaurant menu is a good mental model. Five breads CROSS JOIN four curries gives twenty possible thalis. That is useful when you are printing the menu, and a disaster if it happens to your order table by accident.

Store x day grid with zero-filled revenue (SQL Server 2022+) T-SQL

```
WITH Days AS (
    SELECT CAST(DATEADD(DAY, -value, GETDATE()) AS date) AS SaleDay
    FROM GENERATE_SERIES(0, 29)
)
SELECT d.SaleDay, s.StoreID,
       ISNULL(SUM(o.Amount), 0) AS Revenue
FROM Days AS d
CROSS JOIN dbo.Stores AS s
LEFT JOIN dbo.Orders AS o
       ON o.StoreID = s.StoreID
      AND o.OrderDate = d.SaleDay
GROUP BY d.SaleDay, s.StoreID;
```

PART 1 · JOIN FOUNDATIONS & NULL LOGIC

04 COALESCE, NULLIF, ISNULL and IS DISTINCT FROM

Easy | NULL handling | Asked at: TCS, Infosys, Cognizant, Wipro

THE INTERVIEWER ASKS

You have two columns. How do you check whether they are different, treating NULL as just another value to compare?

ANSWER IN 30 SECONDS

`a <> b` returns UNKNOWN when either side is NULL, so the row is silently dropped. In SQL Server 2022 and later, use `a IS DISTINCT FROM b` for NULL-safe comparison. Use COALESCE to substitute a default and NULLIF to turn a sentinel value, such as zero, into NULL.

BUILD THE ANSWER STEP BY STEP

1. Show the trap: `WHERE OldEmail <> NewEmail` excludes rows where either email is NULL, so a change from NULL to a real email is never detected.

2. Give the SQL Server 2022 answer: `IS DISTINCT FROM`. NULL vs a value is distinct; NULL vs NULL is not distinct.
3. Give the pre-2022 equivalent using EXISTS with EXCEPT, which treats NULLs as equal (see code).
4. `COALESCE(a, b, c)` returns the first non-NULL argument. `NULLIF(a, b)` returns NULL when a equals b, which is the standard divide-by-zero guard.

EDGE CASES THAT IMPRESS

- `ISNULL` takes the data type of its **first** argument. `ISNULL(@code_char2, 'UNKNOWN')` silently truncates to 'UN'. COALESCE uses data type precedence instead.
- COALESCE is rewritten internally as a CASE expression, so a subquery inside it can be evaluated more than once.
- `ISNULL` produces a NOT NULL computed column; COALESCE does not. This matters when you want to index or persist a computed column.

AVOID SAYING

- "NULL = NULL is true." It is UNKNOWN, and WHERE treats UNKNOWN as not true.
- Saying ISNULL and COALESCE are identical. Type handling and evaluation differ.

Likely follow-up: Find customers whose email changed, including NULL to value and value to NULL.

On SQL Server 2022: `WHERE OldEmail IS DISTINCT FROM NewEmail`. On older versions: `WHERE EXISTS (SELECT OldEmail EXCEPT SELECT NewEmail)`.

PRO TIP

In SQL Server, `SET ANSI_NULLS OFF` is deprecated. Never rely on it to make `= NULL` work. Always use IS NULL, IS NOT NULL or IS DISTINCT FROM.

NULL-safe change detection and divide-by-zero guard T-SQL

```sql
-- SQL Server 2022+
SELECT CustomerID
FROM dbo.CustomerChanges
WHERE OldEmail IS DISTINCT FROM NewEmail;

-- Any version: EXCEPT treats NULLs as equal
SELECT CustomerID
FROM dbo.CustomerChanges
WHERE EXISTS (SELECT OldEmail EXCEPT SELECT NewEmail);

-- Divide-by-zero guard (note the 1.0 to avoid integer division)
SELECT OrderID, 1.0 * Amount / NULLIF(Quantity, 0) AS UnitPrice
FROM dbo.OrderItems;
```

PART 1 · JOIN FOUNDATIONS & NULL LOGIC

05 Subquery in SELECT vs FROM vs WHERE

Medium | Subqueries | Asked at: TCS, Razorpay, Swiggy, Target

THE INTERVIEWER ASKS

A subquery can appear in the SELECT list, in the FROM clause, or in the WHERE clause. What is the difference, and when do you use each?

ANSWER IN 30 SECONDS

A SELECT-list subquery returns one scalar value per row and is usually correlated. A FROM-clause subquery (derived table) produces a result set you can join to. A WHERE-clause subquery filters, using EXISTS, IN or a scalar comparison.

BUILD THE ANSWER STEP BY STEP

1. SELECT list: `(SELECT COUNT(*) FROM dbo.Orders o WHERE o.CustomerID = c.CustomerID) AS OrderCount`.
2. FROM clause: `FROM (SELECT CustomerID, SUM(Amount) AS Total FROM dbo.Orders GROUP BY CustomerID) AS t`.
3. WHERE clause: `WHERE CustomerID IN (SELECT CustomerID FROM dbo.Customers WHERE State = 'Rajasthan')`.
4. Performance view: scalar subqueries are evaluated per outer row unless the optimizer rewrites them; derived tables are evaluated as part of one plan; EXISTS and IN in WHERE are usually turned into semi-joins.
5. Mention CTEs as the readable alternative to nested derived tables.

EDGE CASES THAT IMPRESS

- A scalar subquery that returns more than one row raises error 512 at run time.
- SQL Server requires an alias on every derived table. Without one you get a syntax error.
- SQL Server never materialises a CTE. If a CTE is referenced twice, its query runs twice (see Question 24).

AVOID SAYING

- "They are all the same with different syntax." Semantics and plans differ.
- Nesting five levels deep when a chain of CTEs would read cleanly.

Likely follow-up: Rewrite a scalar subquery in the SELECT list as a join.

Pre-aggregate in a derived table (or CTE) grouped by CustomerID, then LEFT JOIN it to Customers and use ISNULL for customers with no orders.

Same result, two shapes T-SQL

```
-- Scalar subquery in SELECT
SELECT c.CustomerID,
       (SELECT COUNT(*) FROM dbo.Orders AS o
        WHERE o.CustomerID = c.CustomerID) AS OrderCount
FROM dbo.Customers AS c;

-- Derived table in FROM
SELECT c.CustomerID, ISNULL(t.OrderCount, 0) AS OrderCount
FROM dbo.Customers AS c
LEFT JOIN (SELECT CustomerID, COUNT(*) AS OrderCount
           FROM dbo.Orders GROUP BY CustomerID) AS t
       ON t.CustomerID = c.CustomerID;
```

PART 1 · JOIN FOUNDATIONS & NULL LOGIC

06 Correlated Subquery: What and Why

Medium | Subqueries | Asked at: TCS, Infosys, Wells Fargo, Target

THE INTERVIEWER ASKS

> What is a correlated subquery? Give an example and describe its performance.

ANSWER IN 30 SECONDS

A correlated subquery references a column from the outer query, so logically it is re-evaluated for every outer row, like a loop. SQL Server often rewrites correlated EXISTS, IN and simple aggregates into joins, but you should confirm in the execution plan rather than assume.

BUILD THE ANSWER STEP BY STEP

1. Define correlation: the inner query depends on a value supplied by the outer row.

2. Classic example: for each customer, the date of their most recent order, computed with a subquery in the SELECT list.
3. Explain the cost model. If the optimizer cannot decorrelate it, a million customers means a million inner executions, typically shown as a Nested Loops join with the inner side executed once per outer row.
4. Offer rewrites: a window function with ROW_NUMBER, a grouped derived table joined back, or OUTER APPLY.

EDGE CASES THAT IMPRESS

- A correlated scalar subquery that returns more than one row raises error 512. Guard it with an aggregate or `TOP (1) ... ORDER BY`.
- Inside EXISTS, correlation is the normal idiom and SQL Server handles it well, usually as a Left Semi Join.
- Check the **Number of Executions** property on the inner operator in the actual plan. That number shows whether the loop really happened.

AVOID SAYING

- "Correlated subqueries are always slow." With a small outer set or a good index on the inner table they are fine.
- Confusing correlated with nested. Every correlated subquery is nested; not every nested subquery is correlated.

Likely follow-up: Rewrite the latest-order-per-customer query without a correlated subquery.

Use `ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC)` in a CTE and keep `rn = 1`, or simply GROUP BY CustomerID with MAX(OrderDate) if only the date is needed.

Correlated vs grouped T-SQL

```
-- Correlated: logically one inner lookup per customer
SELECT c.CustomerID,
       (SELECT MAX(o.OrderDate) FROM dbo.Orders AS o
        WHERE o.CustomerID = c.CustomerID) AS LastOrder
FROM dbo.Customers AS c;

-- Set-based rewrite
SELECT c.CustomerID, t.LastOrder
FROM dbo.Customers AS c
LEFT JOIN (SELECT CustomerID, MAX(OrderDate) AS LastOrder
           FROM dbo.Orders GROUP BY CustomerID) AS t
       ON t.CustomerID = c.CustomerID;
```

02

Existence Checks & Advanced Joins

Semi-joins, anti-joins, APPLY and the optimizer's freedom to reorder your joins.

Q07	Anti-Join and the NOT IN NULL Trap	Medium
Q08	EXISTS vs IN vs JOIN	Medium
Q09	Semi-Join	Medium
Q10	CROSS APPLY and OUTER APPLY	Hard
Q11	Does Join Order Matter?	Medium

07

PART 2 · EXISTENCE CHECKS & ADVANCED JOINS

Anti-Join and the NOT IN NULL Trap

Medium | Joins | Asked at: Razorpay, Swiggy, Goldman, JP Morgan

THE INTERVIEWER ASKS

> Return all customers who have never placed an order. Show three different ways to write it and say which one is safest.

ANSWER IN 30 SECONDS

The three forms are `LEFT JOIN ... WHERE o.CustomerID IS NULL`, `NOT EXISTS (...)` and `NOT IN (...)`. The first two are safe. NOT IN is dangerous: if the subquery returns even one NULL, the whole query returns zero rows.

Customer 5 NOT IN (SELECT CustomerID FROM Orders) where the list is (1, 2, NULL)



One NULL in the subquery makes NOT IN return zero rows. NOT EXISTS ignores NULLs and stays correct.

Figure 7. Why a single NULL empties a NOT IN result

BUILD THE ANSWER STEP BY STEP

1. Write all three forms. The interviewer wants proof you have seen each one.
2. Explain why NOT IN breaks. `x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`. The last comparison is UNKNOWN, so the whole expression is never TRUE and every row is filtered out.
3. State a preference for NOT EXISTS. It handles NULLs correctly and SQL Server shows it as a Left Anti Semi Join in the plan.
4. LEFT JOIN with IS NULL is also correct and is handy when you need left-table columns, but it can do extra work matching rows only to throw them away.

EDGE CASES THAT IMPRESS

- If the subquery column is declared NOT NULL, NOT IN is safe. Do not rely on a schema you cannot see.
- NOT EXISTS stops searching the inner table for a given outer row as soon as it finds one match.
- Even on a nullable column with no actual NULLs, SQL Server must plan for the possibility, which can produce extra plan branches for NOT IN.

AVOID SAYING

- "All three are equivalent." They are not when NULLs are present.
- Declaring one form always fastest. It depends on row counts and indexes.

Likely follow-up: What operator does SQL Server use for NOT EXISTS?

A Left Anti Semi Join, implemented as Nested Loops, Hash Match or Merge Join depending on cost. It emits an outer row only when no inner match exists.

PRO TIP

This is the single most common trap I see in mock interviews. If you remember one thing from this section: for "never did X" questions, reach for NOT EXISTS first.

Customers with no orders (safe form) T-SQL

```
SELECT c.CustomerID, c.CustomerName
FROM dbo.Customers AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Orders AS o
    WHERE o.CustomerID = c.CustomerID
);
```

08

PART 2 · EXISTENCE CHECKS & ADVANCED JOINS

EXISTS vs IN vs JOIN

Medium | Joins | Asked at: Flipkart, PhonePe, Walmart Labs, Cred

THE INTERVIEWER ASKS

When would you use EXISTS, IN or a JOIN? Are they always interchangeable?

ANSWER IN 30 SECONDS

They are not interchangeable. A JOIN can multiply rows when the other table has several matches; EXISTS and IN return each outer row at most once. Use EXISTS for a yes/no check, JOIN when you need columns from the other table, and IN for small constant lists or simple uncorrelated subqueries.

BUILD THE ANSWER STEP BY STEP

1. Lead with cardinality. A customer with ten orders appears ten times after a JOIN, but once with EXISTS.
2. EXISTS is a semi-join: for each outer row, it stops scanning the inner side at the first match.
3. For positive checks, SQL Server usually produces the same plan for IN (subquery) and EXISTS. EXISTS is clearer when the subquery is correlated.
4. If someone writes JOIN followed by DISTINCT to remove duplicates, that is usually a sign EXISTS was the right tool.

EDGE CASES THAT IMPRESS

- NOT IN carries the NULL trap from the previous question; IN does not have the same problem for positive matches.
- JOIN plus DISTINCT does more work: it creates the duplicates and then sorts or hashes them away.
- Very long literal IN lists (thousands of values) are better loaded into a temp table or table-valued parameter and joined.

AVOID SAYING

- "EXISTS is always faster than IN." Modern optimizers usually generate the same plan.
- Using a JOIN when you only need a boolean check.

Likely follow-up: Rewrite a JOIN + DISTINCT query using EXISTS.

Keep only the outer table in FROM, move the joined table into a correlated `WHERE EXISTS (SELECT 1 ... WHERE inner.key = outer.key AND <conditions>)`, and remove DISTINCT.

Customers with at least one successful payment T-SQL

```
SELECT c.CustomerID, c.CustomerName
FROM dbo.Customers AS c
WHERE EXISTS (
    SELECT 1
    FROM dbo.Orders   AS o
    JOIN dbo.Payments AS p ON p.OrderID = o.OrderID
    WHERE o.CustomerID = c.CustomerID
      AND p.Status = 'SUCCESS'
);
```

09

PART 2 · EXISTENCE CHECKS & ADVANCED JOINS

Semi-Join

Medium | Joins | Asked at: Goldman, JP Morgan, Razorpay, Zerodha

THE INTERVIEWER ASKS

> What is a semi-join, and why is EXISTS the natural way to write one?

ANSWER IN 30 SECONDS

A semi-join returns rows from the left table that have at least one match on the right, with no duplication and no right-side columns. It answers "does any matching row exist?", which is exactly what EXISTS expresses.

BUILD THE ANSWER STEP BY STEP

1. Contrast with INNER JOIN: ten matches give ten rows with INNER JOIN, but one row with a semi-join.
2. T-SQL has no SEMI JOIN keyword. You write it with EXISTS or IN, and the optimizer recognises the pattern.
3. In the execution plan the operator is labelled **Left Semi Join**. Knowing the name helps you read plans.
4. The mirror image, "no match exists", is the anti-semi-join, written with NOT EXISTS.

EDGE CASES THAT IMPRESS

- DISTINCT after a JOIN gives the same rows but materialises duplicates first.
- Some warehouse engines expose `LEFT SEMI JOIN` syntax; SQL Server does not, so mention it only as background.
- The early exit happens per outer row: once a match is found, the inner search for that row stops.

AVOID SAYING

- "A semi-join is just an INNER JOIN." That misses the no-duplication property.
- Using JOIN with GROUP BY to remove duplicates when EXISTS is cleaner.

Likely follow-up: How do you write an anti-semi-join in standard SQL?

NOT EXISTS, or LEFT JOIN with an IS NULL filter on the right table's key.

Sellers with at least one high-value order T-SQL

```
SELECT s.SellerID, s.BusinessName
FROM dbo.Sellers AS s
WHERE EXISTS (
    SELECT 1
    FROM dbo.Orders AS o
    WHERE o.SellerID = s.SellerID
      AND o.Amount > 100000
);
```

10

PART 2 · EXISTENCE CHECKS & ADVANCED JOINS

CROSS APPLY and OUTER APPLY

Hard | Joins | Asked at: Razorpay, Cred, Walmart Labs, Goldman

THE INTERVIEWER ASKS

> Have you used CROSS APPLY? When would you reach for it?

ANSWER IN 30 SECONDS

APPLY lets the right-hand side reference columns from the current left-hand row, so it is evaluated per left row. It is the SQL Server equivalent of a LATERAL join, and the classic use is "top 3 orders for each customer", where an ordinary derived table cannot see the outer row.

BUILD THE ANSWER STEP BY STEP

1. Core capability: a derived table in FROM cannot reference outer columns. APPLY removes that restriction.
2. CROSS APPLY behaves like an inner join: left rows with no right-side result are dropped. OUTER APPLY behaves like a left join and keeps them with NULLs.
3. Textbook case: `CROSS APPLY (SELECT TOP (3) ... WHERE o.CustomerID = c.CustomerID ORDER BY o.Amount DESC)`.
4. APPLY is also required to call a table-valued function once per row, for example `CROSS APPLY STRING_SPLIT(t.Tags, ',')` or `OPENJSON`.

EDGE CASES THAT IMPRESS

- With an index on (CustomerID, Amount DESC), APPLY + TOP can seek directly to three rows per customer, which beats a full ROW_NUMBER pass when there are few customers and many orders.
- With a huge left side, the per-row execution becomes Nested Loops at scale, and a window function is usually cheaper.
- The inner TOP with ORDER BY is the killer feature: it limits rows per outer row, which a plain TOP cannot do.

AVOID SAYING

- "It's just a subquery." The correlation inside FROM is the whole point.
- Avoiding it out of unfamiliarity. Fluent use of APPLY signals seniority on SQL Server panels.

Likely follow-up: Could you have written this with ROW_NUMBER instead?

Yes, for a fixed N. APPLY is cleaner when the inner logic has its own joins or aggregates, and can be faster with the right index and a small outer set.

PRO TIP

APPLY is one of the few places where SQL Server was ahead of the standard. Oracle and PostgreSQL later added LATERAL for the same idea, so knowing APPLY transfers well.

Top 3 orders per customer — T-SQL

```
SELECT c.CustomerID, t.OrderID, t.Amount
FROM dbo.Customers AS c
CROSS APPLY (
    SELECT TOP (3) o.OrderID, o.Amount
    FROM dbo.Orders AS o
    WHERE o.CustomerID = c.CustomerID
    ORDER BY o.Amount DESC, o.OrderID
) AS t;
```

11

PART 2 · EXISTENCE CHECKS & ADVANCED JOINS

Does Join Order Matter?

Medium | Joins | Asked at: Goldman, JP Morgan, Wells Fargo, AmEx

THE INTERVIEWER ASKS

> Does the order in which I write joins affect performance?

ANSWER IN 30 SECONDS

For inner joins, usually not. SQL Server's cost-based optimizer is free to reorder them using statistics. Outer joins restrict reordering because moving them changes which rows survive, so the written structure matters there.

BUILD THE ANSWER STEP BY STEP

1. Explain the optimizer's role: it estimates row counts from statistics and explores alternative join orders and physical join types (Nested Loops, Hash Match, Merge Join).
2. Outer joins are not freely reorderable, because they define which rows are preserved.
3. For many-table queries the optimizer cannot explore every order. In the actual plan, the root operator's properties may show **Reason For Early Termination: Time Out**, meaning it settled on a good-enough plan.
4. Practical tip: split very large queries into stages with temp tables so each stage has accurate statistics.

EDGE CASES THAT IMPRESS

- `OPTION (FORCE ORDER)` makes SQL Server keep the written order. Treat it as a last-resort hint.
- Stale statistics are the most common cause of bad join orders. Run `UPDATE STATISTICS` after large data changes.
- Join hints such as `INNER HASH JOIN` also force join order as a side effect, which many developers do not realise.

AVOID SAYING

- "Always join the smallest table first." The optimizer already knows table sizes.
- "Outer join order never matters." It matters semantically.

Likely follow-up: A ten-table query is slow. How do you debug the join order?

Capture the actual execution plan, compare estimated and actual rows on each operator, update statistics where they diverge, check for an optimizer time-out, and consider staging the query into temp tables.

Checking statistics and forcing order (diagnosis only) T-SQL

```
-- How fresh are the statistics?
SELECT s.name, sp.last_updated, sp.rows, sp.modification_counter
FROM sys.stats AS s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) AS sp
WHERE s.object_id = OBJECT_ID('dbo.Orders');

-- Refresh them
UPDATE STATISTICS dbo.Orders WITH FULLSCAN;

-- Last resort: keep the written order
SELECT ... FROM dbo.A JOIN dbo.B ON ... JOIN dbo.C ON ...
OPTION (FORCE ORDER);
```

03

Window Functions

Ranking, running totals and row-to-row comparisons without collapsing a single row.

Q12	ROW_NUMBER vs RANK vs DENSE_RANK	Easy
Q13	PARTITION BY vs GROUP BY	Easy
Q14	Window Function vs Aggregate in the Same Query	Medium
Q15	Running Totals	Easy
Q16	LAG and LEAD	Medium
Q17	Frame Clauses: ROWS vs RANGE	Hard
Q18	The FIRST_VALUE / LAST_VALUE Frame Trap	Hard
Q19	NTILE	Medium
Q20	PERCENT_RANK and CUME_DIST	Hard
Q21	The Filter-Window Trap	Hard
Q22	Top-N per Group	Medium

PART 3 · WINDOW FUNCTIONS

12 ROW_NUMBER vs RANK vs DENSE_RANK

Easy | Window functions | Asked at: Razorpay, Flipkart, Walmart Labs, Cred

THE INTERVIEWER ASKS

> What is the difference between ROW_NUMBER, RANK and DENSE_RANK? Give a case where choosing the wrong one breaks the query.

ANSWER IN 30 SECONDS

ROW_NUMBER gives a unique sequence with no ties (1, 2, 3, 4). RANK gives tied rows the same number and then skips (1, 1, 3, 4). DENSE_RANK gives ties the same number without gaps (1, 1, 2, 3). Using ROW_NUMBER for "top 3 salaries" when salaries tie silently drops valid rows.

Employee	Salary	ROW_NUMBER	RANK	DENSE_RANK
Asha	95,000	1	1	1
Vikram	90,000	2	2	2
Neha	90,000	3	2	2
Imran	82,000	4	4	3
Pooja	80,000	5	5	4

tie

ROW_NUMBER breaks the tie arbitrarily. RANK skips 3. DENSE_RANK has no gap.

Figure 12. Three ranking functions on the same tied salaries

BUILD THE ANSWER STEP BY STEP

1. Lead with tie behaviour. That is the whole question.
2. Walk through the diagram: two employees tie at 90,000. ROW_NUMBER numbers them 1 and 2 in arbitrary order; RANK gives both 1 and the next person 3; DENSE_RANK gives both 1 and the next person 2.
3. Rule of thumb: ROW_NUMBER for pagination and one-row-per-group; DENSE_RANK for "top N salary levels" including ties; RANK when gaps are meaningful, like a competition leaderboard.
4. Trap: `WHERE rnk ⩽ 3` with RANK or DENSE_RANK can return more than three rows. That is correct behaviour, but say it out loud.

EDGE CASES THAT IMPRESS

- ROW_NUMBER with a non-unique ORDER BY is non-deterministic. Two runs can number tied rows differently. Add a tiebreaker such as EmpID.
- RANK's gaps can break downstream code that assumes consecutive numbers.
- All three restart numbering in each PARTITION BY group.

AVOID SAYING

- "They are basically the same." Tied data exposes the difference immediately.
- Using ROW_NUMBER for "top 3 by salary" when the interviewer clearly wants ties included.

Likely follow-up: Top 3 highest-paid employees per department, including ties.

DENSE_RANK partitioned by DeptID ordered by Salary DESC, wrapped in a CTE, keeping rnk <= 3. SQL Server has no QUALIFY clause, so the CTE wrapper is required.

Top 3 salary levels per department T-SQL

```
WITH Ranked AS (
    SELECT EmpID, EmpName, DeptID, Salary,
           DENSE_RANK() OVER (PARTITION BY DeptID
                              ORDER BY Salary DESC) AS rnk
    FROM dbo.Employees
)
SELECT EmpID, EmpName, DeptID, Salary, rnk
FROM Ranked
WHERE rnk ⩽ 3;
```

13

PART 3 · WINDOW FUNCTIONS

PARTITION BY vs GROUP BY

Easy | Window functions | Asked at: TCS, Infosys, Wipro, AmEx

THE INTERVIEWER ASKS

What is the conceptual difference between PARTITION BY in a window function and GROUP BY?

ANSWER IN 30 SECONDS

GROUP BY collapses rows: you get one output row per group. PARTITION BY divides rows into groups for the window function to work over, but every input row stays in the output.

GROUP BY CustomerID

Cust	Amount
C1	500
C1	300
C2	700

→

Cust	SUM
C1	800
C2	700

3 rows in, 2 rows out

SUM() OVER (PARTITION BY CustomerID)

Cust	Amount	CustTotal
C1	500	800
C1	300	800
C2	700	700

3 rows in, 3 rows out; total sits beside each row

Figure 13. GROUP BY collapses rows; PARTITION BY keeps them

BUILD THE ANSWER STEP BY STEP

1. Lead with row preservation. That is the headline difference.
2. Example: `AVG(Amount) OVER (PARTITION BY CustomerID)` shows every order alongside that customer's average. With GROUP BY you would see one row per customer and lose the order detail.
3. They can coexist. You can GROUP BY first and then run a window function over the grouped rows, for example share of the grand total.
4. PARTITION BY without ORDER BY is ideal for partition-wide totals, averages and counts.

EDGE CASES THAT IMPRESS

- A window with an empty `OVER ()` treats the whole result as one partition, useful for percentage-of-total.
- Window functions are evaluated after WHERE, GROUP BY and HAVING. That ordering explains the filter trap in Question 21.
- PARTITION BY usually needs a sort. An index whose key order matches (PARTITION columns, then ORDER BY columns) can remove that sort.

AVOID SAYING

- "PARTITION BY is the same as GROUP BY." Missing the row-preservation point is a red flag.
- Using GROUP BY and then joining back to the detail table when a window function would do it in one pass.

Likely follow-up: Show each order, the customer's total spend, and the order's share of that total in one query.

`SUM(Amount) OVER (PARTITION BY CustomerID)` for the total, and `100.0 * Amount / SUM(Amount) OVER (PARTITION BY CustomerID)` for the share.

Each order with customer total and share T-SQL

```
SELECT OrderID, CustomerID, Amount,
       SUM(Amount) OVER (PARTITION BY CustomerID) AS CustomerTotal,
       CAST(100.0 * Amount
            / SUM(Amount) OVER (PARTITION BY CustomerID)
            AS decimal(5,2))                      AS SharePct
FROM dbo.Orders;
```

PART 3 · WINDOW FUNCTIONS

14 Window Function vs Aggregate in the Same Query

Medium | Window functions | Asked at: Flipkart, Swiggy, JP Morgan India, Zerodha

THE INTERVIEWER ASKS

Return each order's amount alongside the running total for that customer and the customer's grand total, all in one query.

ANSWER IN 30 SECONDS

Use one window function with ORDER BY for the running total and another without ORDER BY for the grand total. An aggregate with GROUP BY collapses rows; a window function keeps every row and computes alongside it.

BUILD THE ANSWER STEP BY STEP

1. `SUM(Amount)` with GROUP BY gives one row per customer. `SUM(Amount) OVER (PARTITION BY CustomerID)` attaches the same total to every row of that customer.
2. Adding `ORDER BY OrderDate` inside OVER turns a total into a running total.
3. Put three columns side by side: the row amount, the running total and the customer total.
4. SQL Server also lets you nest an aggregate inside a window function after GROUP BY: `SUM(SUM(Amount)) OVER ()` gives the grand total across groups.

EDGE CASES THAT IMPRESS

- When ORDER BY is present and no frame is written, SQL Server uses `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. Ties on OrderDate get the same running total. Write ROWS explicitly.
- SQL Server's RANGE frame only supports UNBOUNDED and CURRENT ROW. `RANGE BETWEEN INTERVAL '6' DAY PRECEDING` does not exist in T-SQL, so a true 7-day rolling window needs daily rows first (see code).
- Mixing GROUP BY and OVER is legal; the window runs on the grouped result.

AVOID SAYING

- "Window functions are just a fancy GROUP BY." They return a different shape.
- "Running totals need a self-join." That is the pre-2012 approach.

Likely follow-up: What does the default frame look like, and how would you change it to a 7-day moving average?

Default is RANGE from the partition start to the current row. For a moving average, aggregate to one row per day, then use `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`.

Running total, grand total and 7-day average T-SQL

```sql
SELECT OrderID, CustomerID, OrderDate, Amount,
       SUM(Amount) OVER (PARTITION BY CustomerID ORDER BY OrderDate, OrderID
                         ROWS UNBOUNDED PRECEDING)       AS RunningTotal,
       SUM(Amount) OVER (PARTITION BY CustomerID)        AS CustomerTotal
FROM dbo.Orders
ORDER BY CustomerID, OrderDate;

-- 7-day moving average: one row per day first
WITH Daily AS (
    SELECT OrderDate, SUM(Amount) AS DayAmount
    FROM dbo.Orders GROUP BY OrderDate
)
SELECT OrderDate, DayAmount,
       AVG(DayAmount) OVER (ORDER BY OrderDate
                            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS Avg7d
FROM Daily;
```

PART 3 · WINDOW FUNCTIONS

15 Running Totals

Easy | Window functions | Asked at: TCS, Infosys, Razorpay, Swiggy

THE INTERVIEWER ASKS

> Compute the running total of daily revenue.

ANSWER IN 30 SECONDS

`SUM(Revenue) OVER (ORDER BY SaleDate ROWS UNBOUNDED PRECEDING)`. With ORDER BY inside OVER, the frame accumulates from the start of the partition to the current row.

BUILD THE ANSWER STEP BY STEP

1. Give the one-liner immediately. This is a short-answer question.
2. Without PARTITION BY the total runs across the whole result. Add `PARTITION BY StoreID` for per-store running totals.
3. Write `ROWS UNBOUNDED PRECEDING` explicitly instead of relying on the default RANGE frame.
4. Variant: running total over the last 7 days only, using `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` on a one-row-per-day table.

EDGE CASES THAT IMPRESS

- With the default RANGE frame, duplicate dates share one running total. ROWS avoids this.
- In SQL Server, ROWS frames typically use an in-memory window spool, while RANGE uses an on-disk worktable. `SET STATISTICS IO ON` shows the worktable reads, so ROWS is usually faster as well as more correct.
- Missing days are skipped. If the report needs every calendar day, CROSS JOIN with a calendar first.

AVOID SAYING

- A correlated subquery (`SELECT SUM(...) WHERE d2 ⩽ d1`). That is quadratic and outdated since SQL Server 2012.
- Omitting ORDER BY. Without it, SUM returns the partition total on every row.

Likely follow-up: Make the running total reset every month.

Partition by month: `PARTITION BY DATETRUNC(month, SaleDate)` on SQL Server 2022, or `PARTITION BY YEAR(SaleDate), MONTH(SaleDate)` on older versions.

Running total, resetting each month T-SQL

```
SELECT SaleDate, Revenue,
       SUM(Revenue) OVER (ORDER BY SaleDate
                          ROWS UNBOUNDED PRECEDING)       AS RunningTotal,
       SUM(Revenue) OVER (PARTITION BY YEAR(SaleDate), MONTH(SaleDate)
                          ORDER BY SaleDate
                          ROWS UNBOUNDED PRECEDING)       AS MonthToDate
FROM dbo.DailyRevenue;
```

PART 3 · WINDOW FUNCTIONS

16 LAG and LEAD

Medium | Window functions | Asked at: Razorpay, Swiggy, Zerodha, Cred

THE INTERVIEWER ASKS

> Explain LAG and LEAD. Show how you would compute month-over-month revenue change.

ANSWER IN 30 SECONDS

LAG returns a value from an earlier row in the partition; LEAD returns a value from a later row. Both take an offset and a default value for when that row does not exist, which matters for the first or last row.

BUILD THE ANSWER STEP BY STEP

1. Signature: `LAG(column, offset, default) OVER (PARTITION BY ... ORDER BY ...)`. Offset defaults to 1, default value to NULL.
2. Month-over-month change: `Revenue - LAG(Revenue) OVER (ORDER BY MonthStart)`.
3. LEAD is the mirror image, useful for "time until next event" such as the gap between a customer's orders.
4. Decide deliberately what the first month should show. A default of 0 makes the first change equal to the full revenue, which may mislead; NULL is often more honest.

EDGE CASES THAT IMPRESS

- Integer division trap: if Revenue is an int column, `(cur - prev) / prev` truncates to whole numbers. Multiply by 1.0 or CAST to decimal.
- LAG and LEAD do not accept a frame clause. They address a specific row, not a range.
- Without PARTITION BY they operate across the whole result, which is right for a single time series.

AVOID SAYING

- Simulating LAG with a self-join on `RowNum - 1`. It is slow and hard to read.
- Forgetting the first row, where LAG returns the default.

Likely follow-up: Now give the percentage change instead of the absolute change.

`100.0 * (Revenue - LAG(Revenue) OVER (...)) / NULLIF(LAG(Revenue) OVER (...), 0)`. NULLIF guards against division by zero and 100.0 forces decimal arithmetic.

Month-over-month change T-SQL

```
SELECT MonthStart, Revenue,
       Revenue - LAG(Revenue) OVER (ORDER BY MonthStart) AS MoMChange,
       CAST(100.0 * (Revenue - LAG(Revenue) OVER (ORDER BY MonthStart))
            / NULLIF(LAG(Revenue) OVER (ORDER BY MonthStart), 0)
            AS decimal(6,2))                              AS MoMPct
FROM dbo.MonthlyRevenue;
```

PART 3 · WINDOW FUNCTIONS

17 Frame Clauses: ROWS vs RANGE

Hard | Window functions | Asked at: Goldman, Razorpay, Walmart Labs, JP Morgan

THE INTERVIEWER ASKS

> Walk me through frame clauses. What is the difference between ROWS BETWEEN and RANGE BETWEEN, and what does UNBOUNDED PRECEDING mean?

ANSWER IN 30 SECONDS

ROWS counts physical rows. RANGE treats all rows with the same ORDER BY value as one block. UNBOUNDED PRECEDING means "from the first row of the partition". When you write ORDER BY without a frame, SQL Server uses RANGE UNBOUNDED PRECEDING to CURRENT ROW, which silently includes tied rows.

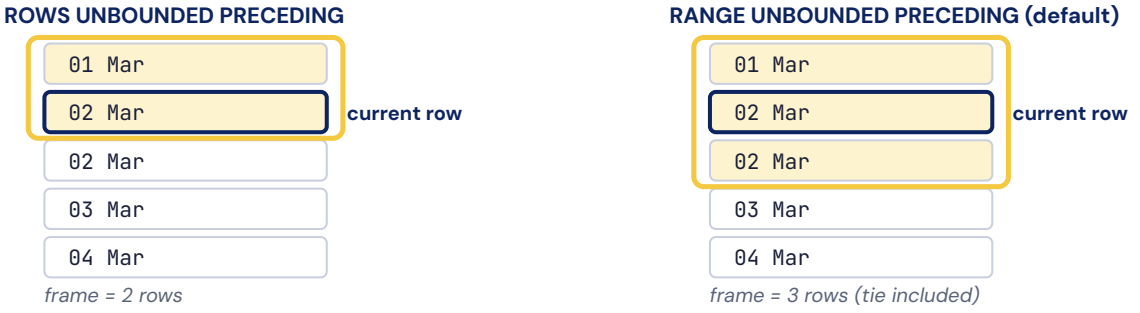


Figure 17. ROWS vs RANGE when two rows share the same date

BUILD THE ANSWER STEP BY STEP

1. A frame is the set of rows the aggregate sees for the current row, anchored on that row.
2. Walk through the diagram: two rows share 2 March. With ROWS, the current row's frame stops at itself. With RANGE, it extends to include its tie.
3. Vocabulary: UNBOUNDED PRECEDING (partition start), UNBOUNDED FOLLOWING (partition end), CURRENT ROW, and `n PRECEDING` / `n FOLLOWING` (ROWS only in SQL Server).
4. Trap: the default RANGE frame makes duplicate dates jump together in a running total. Specify ROWS.

EDGE CASES THAT IMPRESS

- Without ORDER BY, the frame is the entire partition, which is what you want for share-of-total.
- SQL Server supports `n PRECEDING` only with ROWS. RANGE accepts UNBOUNDED and CURRENT ROW only.
- ROWS frames can use an in-memory spool; RANGE frames use an on-disk worktable. Specifying ROWS is often a measurable speed-up.

AVOID SAYING

- Using RANGE for moving averages. Ties lump rows together.
- Assuming the default is ROWS. It is RANGE.

Likely follow-up: Write a 7-day moving sum of daily transaction volume.

`SUM(Volume) OVER (ORDER BY TxnDate ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`, on a table that has one row per day.

PRO TIP

In my classes I make students write the frame every time for two weeks. After that, they never get bitten by the default again.

Rolling 7-day sum T-SQL

```
SELECT TxnDate, Volume,
       SUM(Volume) OVER (ORDER BY TxnDate
                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS Rolling7d
FROM dbo.DailyTransactions;
```

PART 3 · WINDOW FUNCTIONS

18 The FIRST_VALUE / LAST_VALUE Frame Trap

Hard | Window functions | Asked at: Goldman, Walmart Labs, JP Morgan, AmEx

THE INTERVIEWER ASKS

If I write `LAST_VALUE(Amount) OVER (PARTITION BY CustomerID ORDER BY OrderDate)`, what value do I actually get?

ANSWER IN 30 SECONDS

Not the customer's last order. Because the default frame ends at the current row (and its ties), LAST_VALUE returns the last value seen so far, which is normally the current row. To get the true last value, specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

BUILD THE ANSWER STEP BY STEP

1. State the trap first. Interviewers ask it because most candidates fall for it.
2. Explain why: with ORDER BY the default frame is "partition start to current row", so the frame's last row is the current row or its last tie.
3. Fix: write the full frame explicitly.
4. FIRST_VALUE works with the default frame, because the first row of "start to current" is always the partition's first row.

EDGE CASES THAT IMPRESS

- An alternative is FIRST_VALUE with the ORDER BY reversed (DESC). Many people find it cleaner.
- SQL Server 2022 added `IGNORE NULLS` for FIRST_VALUE and LAST_VALUE, handy for carrying the last known value forward.
- With duplicate OrderDates and the default RANGE frame, LAST_VALUE can return a tie's value rather than the current row's, which looks random.

AVOID SAYING

- "LAST_VALUE returns the partition's last row" without the frame caveat.
- Dropping the frame to keep the query short.

Likely follow-up: Get the first and last order amount per customer in one query.

Two window expressions: FIRST_VALUE with the default frame, and LAST_VALUE with ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING (or FIRST_VALUE ordered DESC).

First and last order amount per customer T-SQL

```
SELECT CustomerID, OrderDate, Amount,
       FIRST_VALUE(Amount) OVER (PARTITION BY CustomerID
                                 ORDER BY OrderDate, OrderID) AS FirstAmt,
       LAST_VALUE(Amount)  OVER (PARTITION BY CustomerID
                                 ORDER BY OrderDate, OrderID
                                 ROWS BETWEEN UNBOUNDED PRECEDING
                                          AND UNBOUNDED FOLLOWING) AS LastAmt
FROM dbo.Orders;
```

19

PART 3 · WINDOW FUNCTIONS

NTILE

Medium | Window functions | Asked at: Walmart Labs, Target, AmEx, Wells Fargo

THE INTERVIEWER ASKS

What does NTILE do? Will it always give equal-sized buckets?

ANSWER IN 30 SECONDS

NTILE(n) divides the ordered partition into n buckets of approximately equal size and labels each row with its bucket number. Buckets are equal only when the row count divides evenly; otherwise the first buckets get one extra row each.

BUILD THE ANSWER STEP BY STEP

1. Definition: `NTILE(4) OVER (ORDER BY Amount)` assigns quartiles.
2. Uneven case: 10 rows into 4 buckets gives sizes 3, 3, 2, 2. The extra rows go to the lowest-numbered buckets.
3. Uses: customer deciles by spend, splitting a list into test and control groups, batching work into n chunks.
4. Trap: NTILE splits by row position, not by value. Two rows with the same amount can land in different buckets.

EDGE CASES THAT IMPRESS

- With PARTITION BY, bucket numbering restarts in each partition.
- If ties must share a bucket, compute PERCENT_RANK or CUME_DIST and bucket with CASE or CEILING instead.
- If a partition has fewer rows than buckets, the higher bucket numbers are simply not used.

AVOID SAYING

- "NTILE always gives exactly equal buckets."
- Treating NTILE buckets as value-based percentiles. They are row-count based.

Likely follow-up: Assign customers to deciles by lifetime spend.

Aggregate spend per customer, then `NTILE(10) OVER (ORDER BY TotalSpend DESC)` so decile 1 holds the top spenders.

Customer spend deciles T-SQL

```
WITH Spend AS (
    SELECT CustomerID, SUM(Amount) AS TotalSpend
    FROM dbo.Orders GROUP BY CustomerID
)
SELECT CustomerID, TotalSpend,
       NTILE(10) OVER (ORDER BY TotalSpend DESC) AS SpendDecile
FROM Spend;
```

20

PART 3 · WINDOW FUNCTIONS

PERCENT_RANK and CUME_DIST

Hard | Window functions | Asked at: Goldman, JP Morgan, Walmart Labs, Target

THE INTERVIEWER ASKS

What is the difference between PERCENT_RANK and CUME_DIST?

ANSWER IN 30 SECONDS

Both return a value between 0 and 1 describing position in the partition. PERCENT_RANK is (RANK - 1) / (rows - 1), the share of rows strictly below the current one, starting at 0. CUME_DIST is (rows with value <= current) / rows, so it is always greater than 0.

BUILD THE ANSWER STEP BY STEP

1. Write the formulas. Interviewers want the maths, not hand-waving.
2. PERCENT_RANK answers "what fraction of rows are below me?". CUME_DIST answers "what fraction are at or below me?".
3. Worked example with values 10, 20, 20, 30. For 20: PERCENT_RANK = (2 - 1) / (4 - 1) = 0.333; CUME_DIST = 3 / 4 = 0.75.
4. Related SQL Server functions: PERCENTILE_CONT and PERCENTILE_DISC calculate the median or any percentile, and in T-SQL they require an OVER clause.

EDGE CASES THAT IMPRESS

- Tied values receive the same result in both functions.
- PERCENT_RANK for a single-row partition is 0, not NULL.
- CUME_DIST never decreases as you move down the ORDER BY, a useful sanity check.

AVOID SAYING

- Confusing them with NTILE, which returns bucket numbers, not fractions.
- "They're basically the same." Inclusion of the current row changes the answer.

Likely follow-up: Where would you use PERCENT_RANK in a real report?

Percentile bands for salaries, exam scores or seller performance, for example flagging sellers in the bottom 10 percent for review.

Position of each employee's salary T-SQL

```
SELECT EmpName, Salary,
       PERCENT_RANK() OVER (ORDER BY Salary) AS PctRank,
       CUME_DIST()    OVER (ORDER BY Salary) AS CumeDist,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary)
                      OVER ()                AS MedianSalary
FROM dbo.Employees;
```

21

PART 3 · WINDOW FUNCTIONS

The Filter-Window Trap

Hard | Window functions | Asked at: Razorpay, Cred, Goldman, Flipkart

THE INTERVIEWER ASKS

> Why can't I write `WHERE ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC) = 1`? What is the right way?

ANSWER IN 30 SECONDS

WHERE is evaluated before window functions, so the window result does not exist yet. SQL Server raises error 4108: windowed functions can only appear in the SELECT or ORDER BY clauses. Compute the window in a CTE or derived table and filter in the outer query.



Filter on a window result in an outer query: CTE or derived table, then WHERE rn = 1.

Figure 21. Logical query processing order in SQL Server

BUILD THE ANSWER STEP BY STEP

1. Walk through logical processing order (diagram): FROM, WHERE, GROUP BY, HAVING, SELECT (window functions here), DISTINCT, ORDER BY, TOP/OFFSET.
2. Portable fix: CTE with ROW_NUMBER as a column, then `WHERE rn = 1` outside.
3. QUALIFY exists in Snowflake, BigQuery, Teradata and Databricks, but not in SQL Server. Say this clearly on a SQL Server panel.
4. HAVING also runs before window functions, so it has the same restriction.

EDGE CASES THAT IMPRESS

- In SQL Server, a CTE and a derived table produce the same plan. There is no optimisation fence.
- Window functions are allowed in ORDER BY because ORDER BY runs after SELECT.
- Shortcut for latest-per-group: `SELECT TOP (1) WITH TIES ... ORDER BY ROW_NUMBER() OVER (...)` works but is harder to read and often slower.

AVOID SAYING

- "Just put it in WHERE." The interviewer wants you to explain why it fails.
- Writing QUALIFY on a SQL Server whiteboard.

Likely follow-up: Compare filtering with WHERE, HAVING and QUALIFY.

WHERE filters rows before grouping, HAVING filters groups after aggregation, and QUALIFY (where it exists) filters after window functions. In SQL Server, the QUALIFY step is done with a CTE.

Latest order per customer T-SQL

```
WITH Ranked AS (
    SELECT OrderID, CustomerID, OrderDate, Amount,
           ROW_NUMBER() OVER (PARTITION BY CustomerID
                              ORDER BY OrderDate DESC, OrderID DESC) AS rn
    FROM dbo.Orders
)
SELECT OrderID, CustomerID, OrderDate, Amount
FROM Ranked
WHERE rn = 1;
```

22

PART 3 · WINDOW FUNCTIONS

Top-N per Group

Medium | Window functions | Asked at: Razorpay, Swiggy, Flipkart, PhonePe

THE INTERVIEWER ASKS

> Get the three highest-value orders for each customer. Why is a window function the standard answer?

ANSWER IN 30 SECONDS

Number rows within each customer with `ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY Amount DESC)` and keep rows numbered 3 or less. GROUP BY cannot do this because it collapses rows.

BUILD THE ANSWER STEP BY STEP

1. Pattern: number per partition, then filter in an outer query.
2. Choose the ranking function: ROW_NUMBER for exactly three rows, DENSE_RANK if "top three including ties" is meant.
3. Wrap it in a CTE because of the filter-window trap.
4. Mention CROSS APPLY with TOP (3) as the alternative (Question 10).

EDGE CASES THAT IMPRESS

- Ties on Amount with ROW_NUMBER pick arbitrarily. Add OrderID as a tiebreaker.
- The ideal supporting index follows the **POC** pattern: Partition columns, then Order columns, then Covering columns: `(CustomerID, Amount DESC) INCLUDE (OrderID)`. It removes the sort operator.
- Filtering, such as last 90 days, belongs inside the CTE before numbering.

AVOID SAYING

- A correlated COUNT subquery ("count orders larger than mine < 3"). It works but scales badly.
- Putting TOP (3) in a plain subquery. TOP applies to the whole query, not per customer.

Likely follow-up: Top 3 per customer, but only for the last 90 days.

Add `WHERE OrderDate ⩾ DATEADD(DAY, -90, CAST(GETDATE() AS date))` inside the CTE before ROW_NUMBER is computed.

Top 3 orders per customer with supporting index T-SQL

```sql
CREATE INDEX IX_Orders_Customer_Amount
    ON dbo.Orders (CustomerID, Amount DESC)
    INCLUDE (OrderID, OrderDate);

WITH Ranked AS (
    SELECT OrderID, CustomerID, Amount,
           ROW_NUMBER() OVER (PARTITION BY CustomerID
                              ORDER BY Amount DESC, OrderID) AS rn
    FROM dbo.Orders
)
SELECT OrderID, CustomerID, Amount
FROM Ranked
WHERE rn ⩽ 3;
```

04

Modern T-SQL Toolkit

Recursive CTEs, upserts, multi-level totals, JSON, indexed views and partitioning.

Q23	Recursive CTE	Medium
Q24	CTE vs Subquery vs #Temp Table vs @Table Variable	Medium
Q25	GROUPING SETS, ROLLUP and CUBE	Medium
Q26	Upsert with MERGE (and Its Safer Alternative)	Medium
Q27	OUTPUT Clause and Updatable CTEs in DML	Medium
Q28	JSON in SQL Server	Medium
Q29	Indexed Views (SQL Server's Materialized Views)	Medium
Q30	Table Partitioning	Hard

PART 4 · MODERN T-SQL TOOLKIT

23 Recursive CTE

Medium | Modern T-SQL | Asked at: TCS, Walmart Labs, JP Morgan India, Flipkart

THE INTERVIEWER ASKS

> Write a query that returns everyone under a given manager: direct reports, their reports, and so on down the tree.

ANSWER IN 30 SECONDS

Use a recursive CTE. The anchor member selects the starting manager; the recursive member joins the table back to the CTE itself on ManagerID, producing one level per iteration. Add a Depth column to know how far down each person is.

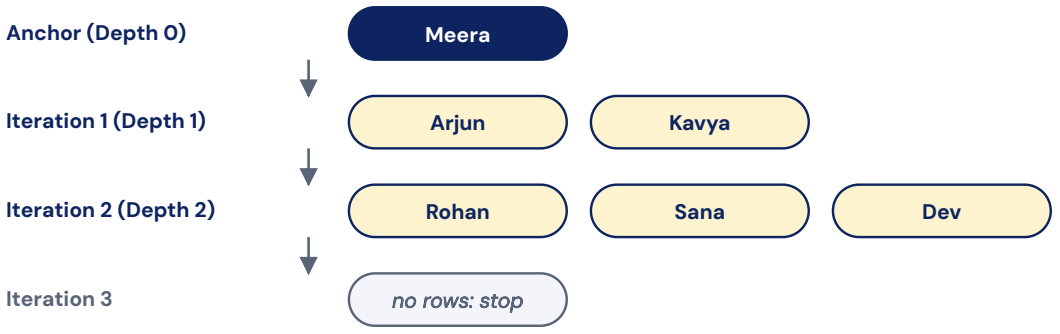


Figure 23. How a recursive CTE walks down the hierarchy one level per pass

BUILD THE ANSWER STEP BY STEP

1. A recursive CTE has two parts joined by UNION ALL: the anchor (base case) and the recursive member.
2. Anchor: the chosen manager, at Depth 0.
3. Recursive member: `JOIN Reports r ON e.ManagerID = r.EmpID`, returning the next level with `r.Depth + 1`.
4. Recursion stops automatically when the recursive member returns no rows.
5. In T-SQL there is no RECURSIVE keyword. You write plain `WITH`, and SQL Server detects the self-reference.

EDGE CASES THAT IMPRESS

- SQL Server stops after **100 levels** by default with error 530. Override with `OPTION (MAXRECURSION n)`; `0` means unlimited, so use it only with a cycle guard.
- Cycles in data cause infinite recursion. Carry a path string and exclude rows already in the path, or cap the depth.
- Only UNION ALL is allowed between anchor and recursive members in SQL Server.

AVOID SAYING

- "I'd write a WHILE loop in a stored procedure." The recursive CTE is the expected answer.
- Forgetting the MAXRECURSION limit on deep hierarchies.

Likely follow-up: How would you show the full reporting path, like "CEO > VP > Manager > You"?

Carry a path column: anchor `CAST(EmpName AS varchar(4000))`, recursive member `r.Path + ' > ' + e.EmpName`. The same column doubles as a cycle check.

Full reporting tree under manager 42 T-SQL

```
WITH Reports AS (
    SELECT EmpID, EmpName, ManagerID, 0 AS Depth,
           CAST(EmpName AS varchar(4000)) AS Path
    FROM dbo.Employees
    WHERE EmpID = 42
    UNION ALL
    SELECT e.EmpID, e.EmpName, e.ManagerID, r.Depth + 1,
           CAST(r.Path + ' > ' + e.EmpName AS varchar(4000))
    FROM dbo.Employees AS e
    JOIN Reports AS r ON e.ManagerID = r.EmpID
    WHERE r.Depth < 20                        -- safety cap
)
SELECT EmpID, EmpName, Depth, Path
FROM Reports
ORDER BY Depth, EmpName
OPTION (MAXRECURSION 50);
```

PART 4 · MODERN T-SQL TOOLKIT

24 CTE vs Subquery vs #Temp Table vs @Table Variable

Medium | Modern T-SQL | Asked at: Flipkart, JP Morgan, Wipro, Cognizant

THE INTERVIEWER ASKS

> When would you choose a CTE, a subquery or a temp table?

ANSWER IN 30 SECONDS

A subquery is inline and single-use. A CTE is the same thing with a name, so it reads better and can be referenced more than once in one statement. A #temp table physically stores the result, gets statistics and can be indexed, so it is the right choice when the result is reused across statements or when you want to break a complex plan into stages.

BUILD THE ANSWER STEP BY STEP

1. Subquery: fine for simple inline logic.
2. CTE: a readability tool. In SQL Server a CTE is **never materialised**. If it is referenced twice, its query is expanded and executed twice.
3. #Temp table: lives in tempdb for the session, supports indexes, and gets column statistics, so the optimizer estimates downstream joins accurately.
4. @Table variable: also in tempdb, but historically no column statistics. Good for small row counts.
5. Rule of thumb: used once, subquery or CTE; used several times or the plan is poor, temp table.

EDGE CASES THAT IMPRESS

- Since SQL Server 2019, table-variable deferred compilation uses the actual row count at first execution, which fixes the old "estimate of 1 row" problem, but table variables still lack column statistics.
- Temp tables can cause recompiles when their row counts change significantly. That is usually helpful.
- Table variables are not affected by a ROLLBACK, which makes them handy for keeping log rows from a failed transaction.

AVOID SAYING

- "CTEs are faster than subqueries." In SQL Server they produce the same plan.
- "Temp tables are slow." They are often the fastest option for reused intermediate results.

Likely follow-up: A CTE is referenced three times and the query is slow. What do you do?
Check the plan: the CTE body appears three times. Materialise it once into a #temp table, add an index if useful, and reference the temp table instead.

Materialising a reused result T-SQL

```
SELECT CustomerID, PlanType
INTO #ActiveCustomers
FROM dbo.Customers
WHERE IsActive = 1;

CREATE CLUSTERED INDEX IX_tmp ON #ActiveCustomers (CustomerID);

SELECT a.PlanType, COUNT(*) AS Orders
FROM #ActiveCustomers AS a
JOIN dbo.Orders AS o ON o.CustomerID = a.CustomerID
GROUP BY a.PlanType;

DROP TABLE IF EXISTS #ActiveCustomers;
```

PART 4 · MODERN T-SQL TOOLKIT

25 GROUPING SETS, ROLLUP and CUBE

Medium | Modern T-SQL | Asked at: Walmart Labs, Target India, AmEx

THE INTERVIEWER ASKS

A report needs total orders by city, by state, and overall, all in one result set and without a pile of UNION ALLs. How?

ANSWER IN 30 SECONDS

Use GROUPING SETS to list exactly the subtotal levels you want. ROLLUP produces a hierarchy of subtotals; CUBE produces every combination. All three run as a single query over the source data.

State	City	Orders	GROUPING_ID
Rajasthan	Jaipur	120	0
Rajasthan	Udaipur	45	0
Rajasthan	All cities	165	1
Gujarat	Surat	80	0
Gujarat	All cities	80	1
All states	All cities	245	3

detail rows (State, City)

state subtotals (State)

grand total ()

Figure 25. ROLLUP (State, City) produces three levels of totals

BUILD THE ANSWER STEP BY STEP

1. `GROUP BY GROUPING SETS ((State, City), (State), ())` gives city detail, state subtotals and a grand total.
2. `GROUP BY ROLLUP (State, City)` produces the same three levels because State > City is a hierarchy.
3. `GROUP BY CUBE (State, Channel)` produces (State, Channel), (State), (Channel) and (). It grows as 2 to the power of the column count, so use it sparingly.
4. `GROUPING(col)` returns 1 when the NULL in that column is a subtotal placeholder, and 0 when it is real data.

EDGE CASES THAT IMPRESS

- NULL ambiguity: a NULL City could be a subtotal or genuinely missing data. GROUPING() tells them apart.
- `GROUPING_ID(State, City)` returns one bitmask integer for the level, handy for sorting subtotal rows.
- SQL Server supports all three in the ISO syntax shown. The older `WITH ROLLUP` and `WITH CUBE` syntax still works but is non-standard.

AVOID SAYING

- "I'd write three queries with UNION ALL." It works, but the interviewer is testing whether you know these features.
- "ROLLUP and CUBE are the same." ROLLUP is hierarchical; CUBE is every combination.

Likely follow-up: How do you label subtotal rows clearly in the output?

`CASE WHEN GROUPING(City) = 1 THEN 'All cities' ELSE City END`, and likewise for State.

City, state and grand totals in one query T-SQL

```
SELECT CASE WHEN GROUPING(c.State) = 1 THEN 'All states' ELSE c.State END AS State,
       CASE WHEN GROUPING(c.City)  = 1 THEN 'All cities' ELSE c.City  END AS City,
       COUNT(*)                        AS Orders,
       GROUPING_ID(c.State, c.City) AS LevelId
FROM dbo.Orders AS o
JOIN dbo.Customers AS c ON c.CustomerID = o.CustomerID
GROUP BY ROLLUP (c.State, c.City)
ORDER BY LevelId, State, City;
```

PART 4 · MODERN T-SQL TOOLKIT

26 Upsert with MERGE (and Its Safer Alternative)

Medium | Modern T-SQL | Asked at: Razorpay, Swiggy, PhonePe, Infosys

THE INTERVIEWER ASKS

> The Customers table has a unique Email. When a sign-up arrives, update LastSeen if the email exists, otherwise insert a new row. How do you write this?

ANSWER IN 30 SECONDS

SQL Server uses MERGE for a single-statement upsert. Under concurrency it needs `WITH (HOLDLOCK)` on the target, otherwise two sessions can both decide the row is missing and one fails with a duplicate-key error. Many teams prefer an explicit UPDATE-then-INSERT inside a serializable transaction.

BUILD THE ANSWER STEP BY STEP

1. MERGE matches a source to a target ON a key, then runs WHEN MATCHED (update), WHEN NOT MATCHED BY TARGET (insert) and optionally WHEN NOT MATCHED BY SOURCE (delete).
2. MERGE must end with a semicolon. Forgetting it is a classic syntax error.
3. Add `WITH (HOLDLOCK)` (serializable range lock) on the target so the existence check and the insert are atomic.
4. Alternative pattern: UPDATE with UPDLOCK and SERIALIZABLE hints; if @@ROWCOUNT = 0, INSERT. Simple, predictable and easy to debug.

EDGE CASES THAT IMPRESS

- MERGE has a long history of reported bugs and surprising behaviour. Careful teams test it thoroughly or avoid it for high-concurrency single-row upserts.
- The OUTPUT clause on MERGE exposes `$action` ('INSERT', 'UPDATE', 'DELETE') so you can log what happened.
- If the source contains the same key twice, MERGE fails with error 8672 because it tried to update a target row more than once. De-duplicate the source first.

AVOID SAYING

- "SELECT first, then INSERT or UPDATE" without locking. Two concurrent calls both see no row and both insert.
- "MERGE is ANSI standard so it is always safe." Concurrency behaviour still needs the hint.

Likely follow-up: What happens if two clients upsert the same email at the same instant?

Without HOLDLOCK, both may evaluate NOT MATCHED and both attempt INSERT; the unique index rejects one with a duplicate-key error. With HOLDLOCK or SERIALIZABLE, the second session waits, then sees the row and updates it.

Concurrency-safe upsert, two ways T-SQL

```
-- 1. MERGE with HOLDLOCK
MERGE dbo.Customers WITH (HOLDLOCK) AS t
USING (SELECT @Email AS Email) AS s
   ON t.Email = s.Email
WHEN MATCHED THEN
    UPDATE SET t.LastSeen = SYSDATETIME()
WHEN NOT MATCHED BY TARGET THEN
    INSERT (Email, LastSeen) VALUES (s.Email, SYSDATETIME());

-- 2. UPDATE, then INSERT if nothing was updated
SET XACT_ABORT ON;
BEGIN TRAN;
    UPDATE dbo.Customers WITH (UPDLOCK, SERIALIZABLE)
       SET LastSeen = SYSDATETIME()
     WHERE Email = @Email;
    IF @@ROWCOUNT = 0
        INSERT dbo.Customers (Email, LastSeen) VALUES (@Email, SYSDATETIME());
COMMIT;
```

PART 4 · MODERN T-SQL TOOLKIT

27 OUTPUT Clause and Updatable CTEs in DML

Medium | Modern T-SQL | Asked at: Razorpay, Cred, Walmart Labs

THE INTERVIEWER ASKS

> Show a useful pattern for modifying data and capturing what changed in a single statement.

ANSWER IN 30 SECONDS

SQL Server's OUTPUT clause returns the `inserted` and `deleted` row images from INSERT, UPDATE, DELETE or MERGE, and `OUTPUT ... INTO` writes them into another table. That gives you archive-and-delete or update-and-audit as one atomic statement. A CTE can also be the target of UPDATE or DELETE, which is the cleanest way to remove duplicates.

BUILD THE ANSWER STEP BY STEP

1. Archive-and-delete: `DELETE ... OUTPUT deleted.* INTO dbo.OrdersArchive`. Both actions succeed or fail together.

2. Update-and-audit: `UPDATE ... OUTPUT deleted.Balance, inserted.Balance INTO dbo.AuditLog` captures before and after values.
3. Updatable CTE: number duplicates with ROW_NUMBER in a CTE, then `DELETE FROM cte WHERE rn > 1`. The delete hits the base table.
4. Composable DML: `INSERT INTO ... SELECT ... FROM (MERGE ... OUTPUT ...) AS x` lets you filter the OUTPUT rows before storing them.

EDGE CASES THAT IMPRESS

- The OUTPUT INTO target cannot have enabled triggers or take part in a foreign key relationship.
- If the table has a trigger, plain OUTPUT (without INTO) to the client fails unless you use OUTPUT INTO a table variable.
- An updatable CTE must resolve to a single base table for the modified columns.

AVOID SAYING

- "You need two statements and a transaction to archive." One statement does it atomically.
- Using `@@IDENTITY` to capture new keys. Use OUTPUT inserted.ID or SCOPE_IDENTITY().

Likely follow-up: How do you capture the identity values of a multi-row INSERT?

`INSERT ... OUTPUT inserted.CustomerID, inserted.Email INTO @NewIds ...`. SCOPE_IDENTITY() only returns the last one.

Archive, audit and de-duplicate T-SQL

```
-- Archive cancelled orders older than 90 days
DELETE FROM dbo.Orders
OUTPUT deleted.* INTO dbo.OrdersArchive
WHERE Status = 'CANCELLED'
  AND OrderDate < DATEADD(DAY, -90, CAST(GETDATE() AS date));

-- Update with an audit trail
UPDATE dbo.Wallets
   SET Balance = Balance - 500
OUTPUT inserted.WalletID, deleted.Balance, inserted.Balance, SYSDATETIME()
  INTO dbo.WalletAudit (WalletID, OldBalance, NewBalance, ChangedAt)
 WHERE WalletID = 101;

-- Remove duplicate customers, keeping the earliest
WITH d AS (
    SELECT ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CustomerID) AS rn
    FROM dbo.Customers
)
DELETE FROM d WHERE rn > 1;
```

28

PART 4 · MODERN T-SQL TOOLKIT

JSON in SQL Server

Medium | Modern T-SQL | Asked at: Cred, Razorpay, Meesho, Zerodha

THE INTERVIEWER ASKS

How does SQL Server handle JSON data, and when would you store JSON instead of proper relational columns?

ANSWER IN 30 SECONDS

SQL Server provides JSON_VALUE (scalar), JSON_QUERY (object or array), OPENJSON (shred into rows), ISJSON (validate) and JSON_MODIFY (update). Traditionally JSON is stored in NVARCHAR(MAX); SQL Server 2025 and Azure SQL add a native json data type. Use JSON for sparse or evolving attributes, never for fields you regularly join, filter or sort on.

BUILD THE ANSWER STEP BY STEP

1. Read a scalar: `JSON_VALUE(Attributes, '$.color')`.
2. Read a nested object or array: `JSON_QUERY(Attributes, '$.sizes')`.
3. Turn an array into rows: `CROSS APPLY OPENJSON(Attributes, '$.sizes')`.
4. Index a JSON property by adding a computed column on JSON_VALUE and indexing that column.
5. Good fits: product attributes that differ by category, webhook payloads, feature-flag configs. Bad fits: anything in a WHERE, JOIN or ORDER BY every day.

EDGE CASES THAT IMPRESS

- Add `CHECK (ISJSON(Attributes) = 1)` so invalid JSON is rejected at write time.
- JSON paths are lax by default: a missing property returns NULL. Prefix with `strict` to raise an error instead.
- JSON_VALUE returns nvarchar(4000). Longer values come back as NULL in lax mode, so use JSON_QUERY or OPENJSON with an explicit schema.

AVOID SAYING

- "JSON makes the schema flexible, so store everything in it." That is how you get slow, untyped tables.
- "JSON in SQL Server can't be indexed." Computed columns make it indexable.

Likely follow-up: How would you make `WHERE JSON_VALUE(Attributes, '$.brand') = 'Tata'` fast?

Add a computed column `Brand AS JSON_VALUE(Attributes, '$.brand')`, index it, and the optimizer can match the expression to the index.

Query and index a JSON property T-SQL

```
ALTER TABLE dbo.Products
    ADD CONSTRAINT CK_Products_Json CHECK (ISJSON(Attributes) = 1);

ALTER TABLE dbo.Products
    ADD Brand AS JSON_VALUE(Attributes, '$.brand');
CREATE INDEX IX_Products_Brand ON dbo.Products (Brand);

SELECT ProductID, ProductName
FROM dbo.Products
WHERE JSON_VALUE(Attributes, '$.brand') = 'Tata';

-- Shred an array into rows
SELECT p.ProductID, s.value AS Size
FROM dbo.Products AS p
CROSS APPLY OPENJSON(p.Attributes, '$.sizes') AS s;
```

29

PART 4 · MODERN T-SQL TOOLKIT

Indexed Views (SQL Server's Materialized Views)

Medium | Modern T-SQL | Asked at: Walmart Labs, Target India, Goldman, Razorpay

THE INTERVIEWER ASKS

> What is a materialized view, how is it different from a regular view, and how does SQL Server keep it fresh?

ANSWER IN 30 SECONDS

A regular view is a stored query that runs every time. SQL Server's materialized view is an **indexed view**: a view WITH SCHEMABINDING that has a unique clustered index, so its result is stored on disk. SQL Server maintains it synchronously on every insert, update and delete, so it is never stale, but every write to the base tables pays the cost.

BUILD THE ANSWER STEP BY STEP

1. Use case: expensive aggregations, such as daily revenue per seller, that dashboards read constantly.
2. Requirements: `WITH SCHEMABINDING`, two-part table names, deterministic expressions, `COUNT_BIG(*)` whenever there is a GROUP BY, and a unique clustered index created first.
3. Refresh model: there is no REFRESH command. The engine updates the view inside the same transaction as the base-table change.
4. Trade-off: fast reads and always-fresh data, paid for with slower writes and some blocking on hot rows.

EDGE CASES THAT IMPRESS

- Restrictions include no OUTER JOIN, no subqueries, no TOP and no AVG. Store SUM and COUNT_BIG and divide when reading.
- Outside Enterprise edition, the optimizer only uses the stored view when you query it with the `WITH (NOEXPAND)` hint.
- If you need scheduled rather than synchronous refresh, build a summary table and reload it with a job instead.

AVOID SAYING

- "Materialized views are always faster." Reads are; writes are slower.
- "You refresh it nightly." Not in SQL Server. It is maintained continuously.

Likely follow-up: How do you avoid write contention on a heavily updated indexed view?

Keep the grouping keys fine-grained so concurrent writes touch different view rows, batch the writes, or switch to a periodically rebuilt summary table if real-time freshness is not required.

Daily revenue per seller as an indexed view T-SQL

```
CREATE VIEW dbo.vDailySellerRevenue
WITH SCHEMABINDING
AS
SELECT o.SellerID,
       o.OrderDate,
       SUM(ISNULL(o.Amount, 0)) AS Revenue,
       COUNT_BIG(*)             AS OrderCount
FROM dbo.Orders AS o
WHERE o.Status = 'PAID'
GROUP BY o.SellerID, o.OrderDate;
GO
CREATE UNIQUE CLUSTERED INDEX UX_vDailySellerRevenue
    ON dbo.vDailySellerRevenue (SellerID, OrderDate);

SELECT SellerID, OrderDate, Revenue
FROM dbo.vDailySellerRevenue WITH (NOEXPAND)
WHERE OrderDate ⩾ '2026-10-01';
```

PART 4 · MODERN T-SQL TOOLKIT

30 Table Partitioning

Hard | Modern T-SQL | Asked at: Flipkart, PhonePe, Walmart Labs, JP Morgan

THE INTERVIEWER ASKS

When does partitioning help, when does it hurt, and how do you set it up in SQL Server?

ANSWER IN 30 SECONDS

Partitioning helps when queries filter on the partition column so whole partitions are skipped (partition elimination), and when old data must be removed instantly by switching or truncating a partition. It hurts when queries span many partitions without filtering on the key. SQL Server natively supports **range** partitioning, built from a partition function and a partition scheme.

Partition function pfOrderMonth (RANGE RIGHT) -> scheme -> dbo.OrdersPartitioned

P1	P2	P3	P4	P5
< Aug 2026	Aug 2026	Sep 2026	Oct 2026	>= Nov 2026

only this partition is read

```
WHERE OrderDate ⩾ '2026-10-01' AND OrderDate < '2026-11-01'
```

TRUNCATE / SWITCH *drops an old month instantly, without a row-by-row DELETE*

Figure 30. Partition elimination on a monthly range-partitioned table

BUILD THE ANSWER STEP BY STEP

1. Partition function: defines boundary values, for example month starts, with RANGE RIGHT or RANGE LEFT.
2. Partition scheme: maps each partition to a filegroup (often all to PRIMARY).
3. Create the table or clustered index ON the scheme, using the partition column.
4. Benefits: partition elimination, `ALTER TABLE ... SWITCH` to archive a month in milliseconds, `TRUNCATE TABLE ... WITH (PARTITIONS (n))`, and per-partition index rebuilds.
5. SQL Server has no LIST or HASH syntax. Emulate list with explicit range boundaries, and hash with a persisted computed column such as `CustomerID % 16`.

EDGE CASES THAT IMPRESS

- RANGE RIGHT is the usual choice for dates: each boundary value belongs to the partition on its right, so '2026-11-01' starts November.
- A unique index on a partitioned table must include the partition column, which affects primary key design.
- Up to 15,000 partitions are supported, but thousands of tiny partitions add overhead. Monthly or daily is typical.
- Partitioning is primarily a data-management feature. It does not automatically make queries faster.

AVOID SAYING

- "Partitioning makes everything faster." Only matching access patterns benefit.
- "Always partition by date." Only if queries and retention work by date.

Likely follow-up: Why might a partitioned table be slower than a non-partitioned one?

If the query does not filter on the partition column, SQL Server must touch every partition, and a seek becomes one seek per partition. Index operations also gain overhead.

Monthly range partitioning T-SQL

```
CREATE PARTITION FUNCTION pfOrderMonth (date)
AS RANGE RIGHT FOR VALUES ('2026-08-01', '2026-09-01', '2026-10-01', '2026-11-01');

CREATE PARTITION SCHEME psOrderMonth
AS PARTITION pfOrderMonth ALL TO ([PRIMARY]);

CREATE TABLE dbo.OrdersPartitioned (
    OrderID   bigint        NOT NULL,
    OrderDate date          NOT NULL,
    Amount    decimal(12,2) NULL,
    CONSTRAINT PK_OrdersPartitioned PRIMARY KEY (OrderID, OrderDate)
) ON psOrderMonth (OrderDate);

-- Remove August instantly
TRUNCATE TABLE dbo.OrdersPartitioned WITH (PARTITIONS (2));
```

05

Indexes & Query Performance

How SQL Server finds rows, why it sometimes refuses your index, and how to read the evidence.

Q31	How a B-tree Index Works	Medium
Q32	Clustered vs Nonclustered Index	Hard
Q33	Composite Index Column Order	Medium
Q34	Covering Index and the INCLUDE Clause	Hard
Q35	Index Seek vs Index Scan vs Table Scan	Medium
Q36	Reading Execution Plans and Query Cost	Hard
Q37	Why Isn't My Index Being Used? (SARGability)	Medium
Q38	Low-Cardinality Columns and Filtered Indexes	Hard
Q39	Hash Index vs B-tree in SQL Server	Hard
Q40	The Cost of Indexes: Maintenance and Write Overhead	Medium

PART 5 · INDEXES & QUERY PERFORMANCE

31 How a B-tree Index Works

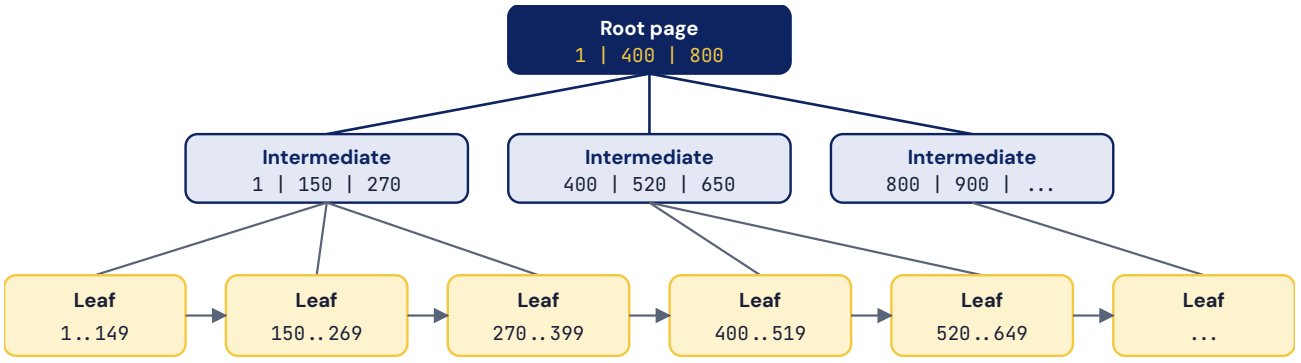
Medium | Indexes | Asked at: Razorpay, Cred, Goldman, Walmart Labs

THE INTERVIEWER ASKS

> Explain how a B-tree index works internally. Why is a lookup O(log n)?

ANSWER IN 30 SECONDS

SQL Server stores an index as a balanced tree of 8 KB pages: a root page, intermediate pages that hold key ranges pointing to child pages, and leaf pages that hold the keys in sorted order. A lookup walks from root to leaf reading one page per level, and the number of levels grows logarithmically with the row count.



Seek for key 455: root -> intermediate (400..649) -> leaf (400..519). Three page reads.
Leaf pages are chained, so a range scan reads sideways after the first seek.

Figure 31. B-tree structure of a SQL Server index (8 KB pages)

BUILD THE ANSWER STEP BY STEP

1. Sketch the levels: root, intermediate, leaf. All leaves are at the same depth, which is what "balanced" means.
2. Explain the depth: each page holds hundreds of key entries, so even hundreds of millions of rows typically need only 3 or 4 levels. A seek therefore costs about 3 or 4 page reads.
3. What the leaf holds depends on the index type: in a clustered index the leaf **is** the data row; in a nonclustered index the leaf holds the key plus a row locator (the clustering key, or a RID on a heap).
4. Range scans: leaf pages are linked to their neighbours, so after seeking to the start of a range SQL Server reads forward along the leaf level.

EDGE CASES THAT IMPRESS

- Inserting into a full page causes a **page split**: half the rows move to a new page, which fragments the index and generates extra log.
- `sys.dm_db_index_physical_stats` reports `index_depth` and fragmentation for any index.
- The B in B-tree stands for balanced, not binary. Each node has hundreds of children, not two.

AVOID SAYING

- "A B-tree is a binary tree."
- "Index lookup is O(1)." That describes a hash index.

Likely follow-up: Why does a B-tree help ORDER BY queries?

Leaves are stored in key order, so SQL Server can read them in sequence and skip the Sort operator entirely. In the plan you see an index scan or seek with Ordered = True and no Sort.

Inspect index depth and page count T-SQL

```
SELECT i.name AS IndexName, ps.index_depth, ps.page_count,
       ps.avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(
        DB_ID(), OBJECT_ID('dbo.Orders'), NULL, NULL, 'DETAILED') AS ps
JOIN sys.indexes AS i
  ON i.object_id = ps.object_id AND i.index_id = ps.index_id
WHERE ps.index_level = 0;
```

PART 5 · INDEXES & QUERY PERFORMANCE

32 Clustered vs Nonclustered Index

Hard | Indexes | Asked at: Razorpay, Goldman, Wells Fargo, JP Morgan

THE INTERVIEWER ASKS

> Explain the difference between a clustered and a nonclustered index. What does a nonclustered index point to in SQL Server?

ANSWER IN 30 SECONDS

A clustered index defines the physical order of the table: its leaf level is the data itself, so a table can have only one. A nonclustered index is a separate B-tree whose leaf holds the index key plus a pointer back to the row. On a clustered table that pointer is the clustering key; on a heap (no clustered index) it is a RID (file, page, slot).

Nonclustered index IX_Orders_CustomerID

CustomerID	OrderID (row locator)
7	1002
7	1450
8	1003

Clustered index PK_Orders (the table)

OrderID	CustomerID	Amount	Status
1002	7	1,250	PAID
1003	8	640	PAID
1450	7	3,900	NEW

Key Lookup per row

SELECT Amount, Status WHERE CustomerID = 7 -> INCLUDE (Amount, Status) removes the lookups.

Figure 32. Nonclustered index seek followed by Key Lookups into the clustered index

BUILD THE ANSWER STEP BY STEP

1. Clustered: table data and index are the same structure, sorted by the clustering key.
2. Nonclustered: separate structure, up to 999 per table. A query that needs columns not in the index performs a **Key Lookup** into the clustered index for each row.
3. By default the PRIMARY KEY creates a clustered index, but you can declare it NONCLUSTERED and cluster on something else.
4. Ideal clustering key: **narrow, unique, static, ever-increasing**, for example an INT or BIGINT IDENTITY.

EDGE CASES THAT IMPRESS

- The clustering key is copied into every nonclustered index, so a wide clustering key bloats all indexes.
- A random key such as `NEWID()` inserts rows all over the B-tree, causing page splits. Prefer IDENTITY or `NEWSEQUENTIALID()`.
- Heaps suffer from **forwarded records**: when an updated row no longer fits, it moves and leaves a forwarding pointer, adding extra reads.

AVOID SAYING

- "A table can have several clustered indexes." Only one.
- "The primary key is always clustered." Default only.

Likely follow-up: Why are short integer clustering keys recommended?

The key is repeated in every nonclustered index leaf and in intermediate levels, so a 4-byte INT keeps every index smaller than a 16-byte GUID or a long composite key.

Primary key clustered vs nonclustered T-SQL

```
-- Typical: narrow, increasing clustered key
CREATE TABLE dbo.Orders (
    OrderID    int IDENTITY(1,1) NOT NULL,
    CustomerID int  NOT NULL,
    OrderDate  date NOT NULL,
    Amount     decimal(12,2) NULL,
    CONSTRAINT PK_Orders PRIMARY KEY CLUSTERED (OrderID)
);

-- Nonclustered index: leaf stores CustomerID + OrderID (the clustering key)
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID
    ON dbo.Orders (CustomerID);
```

PART 5 · INDEXES & QUERY PERFORMANCE

33 Composite Index Column Order

Medium | Indexes | Asked at: TCS, Razorpay, Swiggy, Flipkart

THE INTERVIEWER ASKS

If I create an index on (CustomerID, OrderDate, Status), can SQL Server use it for a query that filters only on OrderDate?

ANSWER IN 30 SECONDS

Not for a seek. A composite index is sorted by the first column, then the second within ties, then the third. It supports seeks on CustomerID, on CustomerID + OrderDate, and on all three, but not on OrderDate alone. SQL Server may still scan the whole index if it is narrower than the table.

Index key order: CustomerID, then OrderDate, then Status

CustomerID	OrderDate	Status
7	2026-09-02	PAID
7	2026-10-05	NEW
8	2026-08-15	PAID
8	2026-10-01	PAID
9	2026-10-03	NEW

WHERE CustomerID = 8

Seek: rows for 8 are contiguous

WHERE OrderDate >= '2026-10-01'

No seek: Oct dates are scattered (rows 2, 4, 5)

Figure 33. The leftmost-prefix rule on a composite index

BUILD THE ANSWER STEP BY STEP

1. Explain the physical sort: like a phone book sorted by surname, then first name. Finding all people named "Rahul" means reading the whole book.
2. State the **leftmost-prefix rule**: seekable predicates must start from the first key column.
3. SQL Server has no index skip scan (unlike Oracle). A query on OrderDate alone becomes an index scan or table scan.
4. Column order guidance: equality columns first, then range columns, then columns used only for ORDER BY.

EDGE CASES THAT IMPRESS

- An index on (CustomerID, OrderDate) serves `WHERE CustomerID = @c ORDER BY OrderDate` with no Sort operator.
- Seek with a range on the first column (`CustomerID > 100`) can only seek on that column; the second column becomes a residual predicate.
- Separate single-column indexes can be combined by the optimizer through index intersection, but a well-designed composite index is almost always better.

AVOID SAYING

- "The index helps any column listed in it." That is the trap.
- Creating one composite index per query. Every index slows writes (Question 40).

Likely follow-up: Some queries filter on (CustomerID, OrderDate) and others only on OrderDate. What do you do?

Either keep (CustomerID, OrderDate) and add a separate index on (OrderDate), or reverse the order if the date-only queries matter more, then check both workloads in Query Store.

Which predicates can seek T-SQL

```
CREATE INDEX IX_Orders_Cust_Date_Status
    ON dbo.Orders (CustomerID, OrderDate, Status);

-- Seek: leading column present
SELECT OrderID FROM dbo.Orders WHERE CustomerID = 7;
SELECT OrderID FROM dbo.Orders WHERE CustomerID = 7 AND OrderDate ⩾ '2026-10-01';

-- No seek on this index: leading column missing
SELECT OrderID FROM dbo.Orders WHERE OrderDate ⩾ '2026-10-01';
```

PART 5 · INDEXES & QUERY PERFORMANCE

34 Covering Index and the INCLUDE Clause

Hard | Indexes | Asked at: Razorpay, Goldman, Walmart Labs, JP Morgan

THE INTERVIEWER ASKS

> What is a covering index? What does INCLUDE do in SQL Server?

ANSWER IN 30 SECONDS

A covering index contains every column a query needs, in its key or its INCLUDE list, so SQL Server answers the query from the index alone with no Key Lookup. INCLUDE adds non-key columns to the leaf level only: they are available to the query but are not part of the sort order.

BUILD THE ANSWER STEP BY STEP

1. Symptom of a non-covering index: the plan shows an Index Seek followed by a **Key Lookup** joined by Nested Loops. With many rows, those lookups dominate the cost.

2. Fix: `CREATE INDEX ... ON dbo.Orders (CustomerID) INCLUDE (Amount, Status)`. Now `SELECT Amount, Status WHERE CustomerID = @c` is covered.
3. Why not put everything in the key? Wider keys mean larger intermediate pages and more write cost. INCLUDE keeps the tree narrow and only widens the leaves.
4. Key limits: up to 32 key columns and 1,700 bytes for a nonclustered key. Included columns do not count against those limits.

EDGE CASES THAT IMPRESS

- Included columns can be types that cannot be key columns, such as nvarchar(max).
- Covering a query that does `SELECT *` means duplicating the table. Cover the columns that hot queries actually use.
- Every included column is updated whenever it changes in the table, so covering indexes cost more on write-heavy tables.

AVOID SAYING

- "A covering index means including every column." Only what the query needs.
- "Covering always wins." Write and storage costs are real.

Likely follow-up: How do you confirm a query is covered?

In the actual execution plan there is an Index Seek or Scan with no Key Lookup or RID Lookup operator. `SET STATISTICS IO ON` also shows fewer logical reads.

Remove a Key Lookup with INCLUDE T-SQL

```
-- Before: Index Seek + Key Lookup
CREATE INDEX IX_Orders_CustomerID ON dbo.Orders (CustomerID);

-- After: fully covered
CREATE INDEX IX_Orders_CustomerID_Cover
    ON dbo.Orders (CustomerID)
    INCLUDE (Amount, Status, OrderDate);

SELECT OrderDate, Amount, Status
FROM dbo.Orders
WHERE CustomerID = 7;
```

PART 5 · INDEXES & QUERY PERFORMANCE

35 Index Seek vs Index Scan vs Table Scan

Medium | Indexes | Asked at: TCS, Wells Fargo, AmEx, Target

THE INTERVIEWER ASKS

> In a SQL Server execution plan, what is the difference between an Index Seek, an Index Scan and a Table Scan?

ANSWER IN 30 SECONDS

A **Seek** navigates the B-tree directly to the matching key range. An **Index Scan** reads all the leaf pages of an index. A **Table Scan** reads every page of a heap; on a clustered table the equivalent is a **Clustered Index Scan**.

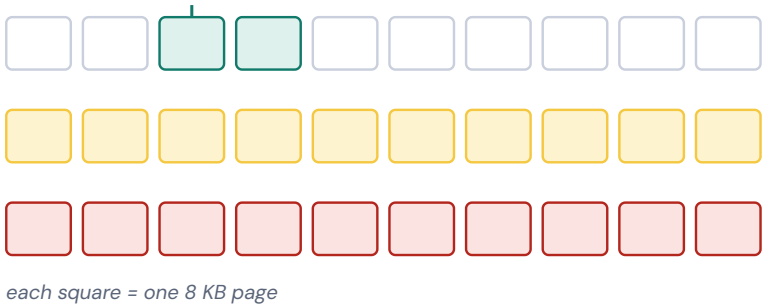


Figure 35. Pages touched by a seek, an index scan and a full scan

BUILD THE ANSWER STEP BY STEP

1. Seek: descends root to leaf, then reads only the qualifying range. Best for selective predicates.
2. Index Scan: reads the whole nonclustered index. Chosen when no seekable predicate exists but the index is narrower than the table.
3. Table Scan / Clustered Index Scan: reads every data page. Chosen when the query needs most of the table or no index helps.
4. A Seek is not automatically good: a seek that returns 90 percent of the table and then does a Key Lookup per row is often worse than a scan.

EDGE CASES THAT IMPRESS

- Seek + Key Lookup means the index found the rows but had to fetch other columns from the clustered index. A covering index removes it.
- A scan with `TOP (1)` may read only one page. Check **Actual Number of Rows Read**, not just the operator name.
- The tipping point where SQL Server prefers a scan over seek + lookups is often a surprisingly small percentage of the table.

AVOID SAYING

- "Table scans are always bad." For small tables or large fractions of a table they are the right plan.
- "A seek is always efficient." Range size and lookups decide.

Likely follow-up: How do you read an actual execution plan quickly?

Start at the right, follow the data flow left, look at the thickest arrows, compare estimated vs actual rows on each operator, and check warning icons (spills, implicit conversions).

See the difference in page reads T-SQL

```
SET STATISTICS IO ON;

-- Seek: a handful of logical reads
SELECT OrderID, Amount FROM dbo.Orders WHERE OrderID = 100250;

-- Clustered index scan: reads every page
SELECT OrderID, Amount FROM dbo.Orders WHERE Amount > 0;

SET STATISTICS IO OFF;
```

36

PART 5 · INDEXES & QUERY PERFORMANCE

Reading Execution Plans and Query Cost

Hard | Performance | Asked at: Razorpay, Swiggy, Goldman, Cred

THE INTERVIEWER ASKS

How do you read an execution plan, and what does "cost" mean in SQL Server?

ANSWER IN 30 SECONDS

The estimated plan shows what the optimizer intends to do; the actual plan also runs the query and records real row counts, executions and warnings. Operator cost and Estimated Subtree Cost are unitless internal estimates, not seconds. The most useful signal is a large gap between estimated and actual rows.

BUILD THE ANSWER STEP BY STEP

1. Estimated plan (Ctrl+L in SSMS) does not run the query. Actual plan (Ctrl+M) runs it, so be careful with UPDATE and DELETE: wrap them in a transaction and roll back.
2. Read right to left: data flows from the leaf operators on the right towards SELECT on the left. Arrow thickness shows row volume.
3. Compare **Estimated Number of Rows** and **Actual Number of Rows**. A 100x or 1000x gap usually means stale statistics, parameter sniffing or a non-sargable predicate.
4. Look for warnings: memory grant spills to tempdb, implicit conversions, missing index suggestions and residual predicates.

EDGE CASES THAT IMPRESS

- Pair the plan with `SET STATISTICS IO, TIME ON` to see logical reads and CPU time per table.
- **Query Store** keeps plan history and runtime stats per query, which makes "it was fast yesterday" investigations possible.
- Missing-index hints are suggestions for one query only. Review them against the whole workload before creating anything.

AVOID SAYING

- "Cost is in milliseconds." It is a relative estimate.
- Tuning without capturing the actual plan and reads first.

Likely follow-up: A query is slow only in production. How do you diagnose it?

Pull its plans from Query Store, compare the fast and slow plans, check for parameter sniffing or a plan change after statistics updated, and compare data distribution and statistics between environments.

Measure before you tune T-SQL

```
SET STATISTICS IO, TIME ON;

SELECT c.City, SUM(o.Amount) AS Revenue
FROM dbo.Orders AS o
JOIN dbo.Customers AS c ON c.CustomerID = o.CustomerID
WHERE o.OrderDate ⩾ '2026-10-01'
GROUP BY c.City;

SET STATISTICS IO, TIME OFF;

-- Slowest queries recorded by Query Store
SELECT TOP (10) q.query_id, rs.avg_duration, rs.count_executions
FROM sys.query_store_runtime_stats AS rs
JOIN sys.query_store_plan  AS p ON p.plan_id = rs.plan_id
JOIN sys.query_store_query AS q ON q.query_id = p.query_id
ORDER BY rs.avg_duration DESC;
```

37

PART 5 · INDEXES & QUERY PERFORMANCE

Why Isn't My Index Being Used? (SARGability)

Medium | Indexes | Asked at: TCS, Infosys, Razorpay, AmEx

THE INTERVIEWER ASKS

There is an index on Email, but `WHERE LOWER(Email) = 'x@y.com'` does a full scan. Why?

ANSWER IN 30 SECONDS

Wrapping the column in a function makes the predicate non-SARGable: SQL Server cannot seek on `LOWER(Email)` because the index is sorted by Email. In SQL Server the fix is often simply to remove LOWER, since the default collations are case-insensitive. Otherwise, index a computed column or rewrite the predicate.

BUILD THE ANSWER STEP BY STEP

1. SARGable means Search ARGument-able: the column must stand alone on one side of the comparison for a seek.
2. Common offenders: `LOWER(col)`, `col + 1 = @x`, `YEAR(OrderDate) = 2026`, `LEFT(Phone, 3) = '987'`, `ISNULL(col, 0) = 0`.
3. Fixes: drop LOWER under a case-insensitive collation; rewrite `YEAR(OrderDate) = 2026` as a range `OrderDate ⩾ '2026-01-01' AND OrderDate < '2027-01-01'`; or create an indexed computed column for the expression.
4. Other reasons an index is skipped: leading wildcard `LIKE '%abc'`, low selectivity, OR conditions across different columns, or a tiny table.

EDGE CASES THAT IMPRESS

- **Implicit conversion** is the hidden killer. A VARCHAR column compared with an NVARCHAR parameter shows `CONVERT_IMPLICIT` in the plan and can turn a seek into a scan. Match parameter types to column types.
- SQL Server has a special case: `CAST(OrderDate AS date) = '2026-10-23'` on a datetime column is still seekable. Most other functions are not.
- `LIKE 'abc%'` can seek; `LIKE '%abc%'` cannot. For contains-style search, look at full-text indexing.

AVOID SAYING

- "The optimizer is wrong" before checking for functions, conversions and wildcards.
- Forcing the index with a hint before understanding why it was skipped.

Likely follow-up: Walk me through debugging an index that isn't being used, end to end.

Capture the actual plan, look for CONVERT_IMPLICIT or functions on the column, check selectivity and statistics freshness, test the rewritten predicate, and only then consider a new or changed index.

Non-SARGable vs SARGable T-SQL

```
-- Scan: function on the column
SELECT OrderID FROM dbo.Orders WHERE YEAR(OrderDate) = 2026;

-- Seek: open-ended range on the bare column
SELECT OrderID FROM dbo.Orders
WHERE OrderDate ⩾ '2026-01-01' AND OrderDate < '2027-01-01';

-- Implicit conversion: VARCHAR column vs NVARCHAR literal
SELECT CustomerID FROM dbo.Customers WHERE Phone = N'9876543210';   -- may scan
SELECT CustomerID FROM dbo.Customers WHERE Phone = '9876543210';    -- seek

-- Index an expression through a computed column
ALTER TABLE dbo.Customers ADD EmailDomain AS SUBSTRING(Email, CHARINDEX('@', Email) + 1, 100);
CREATE INDEX IX_Customers_EmailDomain ON dbo.Customers (EmailDomain);
```

PART 5 · INDEXES & QUERY PERFORMANCE

38 Low-Cardinality Columns and Filtered Indexes

Hard | Indexes | Asked at: Razorpay, Cred, Walmart Labs, AmEx

THE INTERVIEWER ASKS

I have an index on Status, which has only two values, ACTIVE and INACTIVE. The optimizer ignores it. Why? When would a filtered index help?

ANSWER IN 30 SECONDS

With only two values, each one matches a large share of the table, so seeking and then looking up each row costs more than scanning. A **filtered index** with `WHERE Status = 'ACTIVE'` indexes only the rows you care about, making it small and attractive when the filtered value is rare or frequently queried.

BUILD THE ANSWER STEP BY STEP

1. Selectivity: if a predicate matches more than a small fraction of the table, a scan beats seek + lookup.
2. A 50/50 split on Status means an index on Status alone almost never helps.
3. Filtered index: `CREATE INDEX ... ON dbo.Orders (OrderDate) WHERE Status = 'PENDING'`. Only pending orders are stored, so it is tiny and fast.
4. Common pattern: soft deletes, with `WHERE IsDeleted = 0`, or unique-if-not-null constraints with `WHERE Email IS NOT NULL`.

EDGE CASES THAT IMPRESS

- The query predicate must match or imply the filter. `WHERE Status = @status` with a parameter often cannot use it, because the cached plan must work for any value. Use a literal or `OPTION (RECOMPILE)`.
- Filtered indexes require specific SET options (for example ANSI_NULLS and QUOTED_IDENTIFIER ON) for sessions that modify the table.
- A composite index that leads with a low-cardinality column is rarely useful unless queries always filter on it with equality.

AVOID SAYING

- "Index every column." Unused indexes still cost writes (Question 40).
- Blaming the optimizer for skipping a useless index.

Likely follow-up: How do you enforce "email must be unique, but many rows can have no email"?

A unique filtered index: `CREATE UNIQUE INDEX UX_Customers_Email ON dbo.Customers (Email) WHERE Email IS NOT NULL;`

Filtered index for a rare, hot value T-SQL

```
CREATE INDEX IX_Orders_Pending_OrderDate
    ON dbo.Orders (OrderDate)
    INCLUDE (CustomerID, Amount)
    WHERE Status = 'PENDING';

-- Uses the filtered index (literal matches the filter)
SELECT OrderID, CustomerID, Amount
FROM dbo.Orders
WHERE Status = 'PENDING' AND OrderDate < DATEADD(DAY, -2, GETDATE());
```

PART 5 · INDEXES & QUERY PERFORMANCE

39 Hash Index vs B-tree in SQL Server

Hard | Indexes | Asked at: Goldman, JP Morgan, Walmart Labs, Cred

THE INTERVIEWER ASKS

When would you choose a hash index over a B-tree index?

ANSWER IN 30 SECONDS

In SQL Server, hash indexes exist only on **memory-optimized tables** (In-Memory OLTP). They give very fast equality lookups but cannot do ranges, ORDER BY or partial-key searches. Ordinary disk-based tables only have B-tree (rowstore) and columnstore indexes.

BUILD THE ANSWER STEP BY STEP

1. Data structures: a hash index maps a hashed key to a bucket; a B-tree keeps keys sorted.
2. Hash wins only on exact equality on the full key. B-tree handles equality, ranges, prefixes and ordering.
3. On memory-optimized tables, size the hash index with `BUCKET_COUNT`, roughly one to two times the number of distinct keys. Too few buckets create long chains.
4. Memory-optimized tables also support range (Bw-tree) nonclustered indexes for range queries.

EDGE CASES THAT IMPRESS

- A hash index on (A, B) cannot be used for a predicate on A alone. All key columns must be supplied.
- Do not confuse hash **indexes** with the **Hash Match** join and aggregate operators, which build temporary hash tables at run time on any table.
- `sys.dm_db_xtp_hash_index_stats` shows empty buckets and average chain length for tuning BUCKET_COUNT.

AVOID SAYING

- "Hash is always faster than B-tree." Only for exact equality on the whole key.
- "SQL Server has hash indexes on normal tables." It does not.

Likely follow-up: What is the Hash Match operator in a plan?

A run-time operator for joins and aggregations: it builds a hash table from the smaller input in memory and probes it with the larger one. It is unrelated to hash indexes and appears on ordinary tables.

Hash index on a memory-optimized session table T-SQL

```
-- Requires a MEMORY_OPTIMIZED_DATA filegroup in the database
CREATE TABLE dbo.UserSessions (
    SessionID uniqueidentifier NOT NULL
        PRIMARY KEY NONCLUSTERED HASH WITH (BUCKET_COUNT = 2000000),
    CustomerID int        NOT NULL,
    LastSeen   datetime2  NOT NULL,
    INDEX IX_LastSeen NONCLUSTERED (LastSeen)      -- range index
) WITH (MEMORY_OPTIMIZED = ON, DURABILITY = SCHEMA_AND_DATA);
```

40

PART 5 · INDEXES & QUERY PERFORMANCE

The Cost of Indexes: Maintenance and Write Overhead

Medium | Performance | Asked at: Razorpay, Swiggy, Walmart Labs, Goldman

THE INTERVIEWER ASKS

> What is the downside of adding indexes? Why not index every column?

ANSWER IN 30 SECONDS

Every INSERT and DELETE, and every UPDATE that touches an indexed column, must also maintain each affected index. That is write amplification, more transaction log, more locking and more memory. Too many indexes slow writes and push useful data out of the buffer pool.

BUILD THE ANSWER STEP BY STEP

1. Write cost: inserting a row into a table with five nonclustered indexes means six B-tree inserts.
2. Memory: indexes compete with data pages in the buffer pool. A few targeted indexes that stay in memory beat many that do not.
3. Fragmentation: page splits leave half-empty, out-of-order pages. Check `sys.dm_db_index_physical_stats` and choose REORGANIZE (light, online) or REBUILD (full, updates statistics).
4. Process: find slow queries, add the fewest composite indexes that cover them, and regularly drop indexes nobody reads.

EDGE CASES THAT IMPRESS

- `sys.dm_db_index_usage_stats` shows seeks, scans, lookups and updates per index, but it resets when SQL Server restarts. Judge "unused" over a full business cycle.
- For big loads, disabling nonclustered indexes and rebuilding them afterwards is often faster than maintaining them row by row.
- A lower FILLFACTOR leaves room on pages to reduce splits, at the cost of more pages to read.

AVOID SAYING

- "Index everything." The most common junior mistake.
- Ignoring index maintenance until queries fall over.

Likely follow-up: How do you find unused indexes in production?

Query `sys.dm_db_index_usage_stats` joined to `sys.indexes`, looking for indexes with zero seeks, scans and lookups but high updates, measured over a representative period.

Indexes that cost writes but serve no reads T-SQL

```sql
SELECT OBJECT_NAME(i.object_id) AS TableName, i.name AS IndexName,
       us.user_seeks, us.user_scans, us.user_lookups, us.user_updates
FROM sys.indexes AS i
LEFT JOIN sys.dm_db_index_usage_stats AS us
       ON us.object_id = i.object_id AND us.index_id = i.index_id
      AND us.database_id = DB_ID()
WHERE i.type_desc = 'NONCLUSTERED'
  AND OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1
  AND ISNULL(us.user_seeks + us.user_scans + us.user_lookups, 0) = 0
ORDER BY us.user_updates DESC;
```

06

Transactions, Isolation & Concurrency

ACID in practice, the isolation ladder, row versioning, deadlocks and lock escalation.

Q41	ACID Explained Practically	Easy
Q42	Dirty Read vs Non-Repeatable Read vs Phantom Read	Medium
Q43	READ UNCOMMITTED and NOLOCK	Easy
Q44	READ COMMITTED (the Default)	Medium
Q45	REPEATABLE READ	Medium
Q46	SERIALIZABLE	Hard
Q47	Row Versioning: RCSI and SNAPSHOT Isolation	Hard
Q48	Optimistic vs Pessimistic Locking	Medium
Q49	Deadlocks: Detection and Retry	Medium
Q50	Lock Escalation	Hard

41

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

ACID Explained Practically

Easy | Transactions | Asked at: TCS, Infosys, Wipro, Razorpay

THE INTERVIEWER ASKS

> Explain ACID. Don't recite the textbook. Tell me what each letter means when you are writing payment code.

ANSWER IN 30 SECONDS

Atomicity: all statements in the transaction succeed or none do. Consistency: constraints hold before and after. Isolation: concurrent transactions don't see each other's half-finished work. Durability: once COMMIT returns, the change survives a crash because it is in the transaction log on disk.

BUILD THE ANSWER STEP BY STEP

1. Atomicity: in a wallet transfer, the debit and the credit must both land or both roll back.
2. Consistency: database rules (keys, foreign keys, CHECK, NOT NULL) are never violated at commit. Business rules not declared as constraints are the application's job.
3. Isolation: the isolation level decides how much concurrent transactions can see of each other.
4. Durability: SQL Server hardens the log records to disk before acknowledging COMMIT (write-ahead logging).

EDGE CASES THAT IMPRESS

- **SQL Server-specific trap:** without `SET XACT_ABORT ON`, many run-time errors abort only the failing statement, and the transaction stays open with earlier statements still pending commit. Always use XACT_ABORT ON with TRY/CATCH.
- Delayed durability (`DELAYED_DURABILITY = ALLOWED`) trades durability for throughput. A crash can lose the last few committed transactions.
- The C in CAP theorem means replica agreement, a different idea from the C in ACID.

AVOID SAYING

- "Consistency means the data is correct." Too vague. It means declared invariants are preserved.
- "Isolation means transactions are independent." At READ UNCOMMITTED they are anything but.

Likely follow-up: Which ACID property is most often relaxed for performance, and where?

Isolation, by running at READ COMMITTED or using NOLOCK on reports. Durability is sometimes relaxed with delayed durability for high-volume logging tables.

PRO TIP

In SQL Server interviews, mentioning XACT_ABORT unprompted is one of the quickest ways to show real production experience.

Safe wallet transfer template T-SQL

```
SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRAN;
        UPDATE dbo.Wallets SET Balance = Balance - 5000 WHERE WalletID = 101;
        UPDATE dbo.Wallets SET Balance = Balance + 5000 WHERE WalletID = 202;
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;
    THROW;    -- re-raise to the caller
END CATCH;
```

42

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

Dirty Read vs Non-Repeatable Read vs Phantom Read

Medium | Isolation | Asked at: TCS, Cognizant, Target India

THE INTERVIEWER ASKS

> Walk me through dirty reads, non-repeatable reads and phantom reads with a concrete example of each.

ANSWER IN 30 SECONDS

A dirty read sees data another transaction has not committed. A non-repeatable read reads the same row twice and gets different values. A phantom read re-runs the same range query and gets a different set of rows. They are progressively more subtle.

Isolation level (SQL Server)	Dirty read	Non-repeatable	Phantom
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	prevented	possible	possible
REPEATABLE READ	prevented	prevented	possible
SERIALIZABLE	prevented	prevented	prevented
SNAPSHOT	prevented	prevented	prevented

Default is READ COMMITTED (locking on-premises; row versioning when READ_COMMITTED_SNAPSHOT is ON).

Figure 42. Isolation levels and the read anomalies they allow

BUILD THE ANSWER STEP BY STEP

1. Dirty read: T1 changes a balance from 5,000 to 8,000 without committing. T2 reads 8,000 and acts on it. T1 rolls back. T2 used a value that never officially existed.
2. Non-repeatable read: T1 reads an order amount of 500. T2 updates it to 700 and commits. T1 reads again and sees 700.
3. Phantom read: T1 counts today's PLACED orders and gets 50. T2 inserts three more and commits. T1 counts again and gets 53.
4. Map them to levels (see the matrix): READ UNCOMMITTED allows all three; READ COMMITTED prevents dirty reads; REPEATABLE READ also prevents non-repeatable reads; SERIALIZABLE prevents all three.

EDGE CASES THAT IMPRESS

- Phantoms are about which rows are in the set; non-repeatable reads are about a row's values. Interviewers love that distinction.
- **Lost update** is a further anomaly: two transactions both read 100, both add 10, both write 110. One update is lost.
- **Write skew** happens when two transactions read overlapping data and write different rows, breaking a rule together. Only true serializability prevents it.

AVOID SAYING

- Mixing up phantom and non-repeatable reads.
- "Dirty reads are rare." At READ COMMITTED and above they are impossible.

Likely follow-up: Which anomalies can occur at SQL Server's default isolation level?

Default is READ COMMITTED, so non-repeatable reads and phantoms can occur, and so can lost updates unless you use UPDLOCK or an optimistic version check.

Reproducing a phantom read (two sessions) T-SQL

```
-- Session 1
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRAN;
SELECT COUNT(*) FROM dbo.Orders
WHERE OrderDate = CAST(GETDATE() AS date);      -- 120

-- Session 2: insert 5 orders for today and COMMIT

-- Session 1 again
SELECT COUNT(*) FROM dbo.Orders
WHERE OrderDate = CAST(GETDATE() AS date);      -- 125 (phantoms)
COMMIT;
```

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

43 READ UNCOMMITTED and NOLOCK

Easy | Isolation | Asked at: Cognizant, Accenture, Wells Fargo India

THE INTERVIEWER ASKS

> What does READ UNCOMMITTED actually allow? Have you ever used it in production?

ANSWER IN 30 SECONDS

READ UNCOMMITTED, and the equivalent `WITH (NOLOCK)` table hint, lets a query read rows that other transactions have modified but not committed. Readers take no shared locks and are not blocked by writers. The only reasonable use is approximate reporting where speed matters more than accuracy.

BUILD THE ANSWER STEP BY STEP

1. Lowest isolation level: no shared locks, so no waiting on writers.
2. It allows dirty reads, and on SQL Server it can also return the same row twice or skip rows entirely if pages split during an allocation-order scan.
3. Legitimate use: a dashboard tile showing an approximate count, where being off slightly is acceptable.
4. Better alternative in most cases: enable READ_COMMITTED_SNAPSHOT so readers don't block writers and still see only committed data.

EDGE CASES THAT IMPRESS

- NOLOCK scans can fail with error 601, "could not continue scan with NOLOCK due to data movement".
- NOLOCK does not mean "takes no locks at all". It still takes a schema stability (Sch-S) lock, so it can be blocked by schema changes.
- NOLOCK is ignored on the target of an UPDATE or DELETE.

AVOID SAYING

- "It's faster, so use it everywhere." The correctness loss is much bigger than the gain.
- Using NOLOCK on financial reports.

Likely follow-up: Why is a dirty read dangerous in a financial report?

The report can include amounts from transactions that later roll back, or double-count rows that moved during the scan, so the totals may never have been true at any point in time.

Two equivalent ways, and the better alternative T-SQL

```
-- Per-table hint
SELECT COUNT(*) FROM dbo.Orders WITH (NOLOCK) WHERE Status = 'PLACED';

-- Session-level
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT COUNT(*) FROM dbo.Orders WHERE Status = 'PLACED';

-- Better: committed data without blocking (database-level setting)
ALTER DATABASE ShopSmartIndia SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK IMMEDIATE;
```

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

44 READ COMMITTED (the Default)

Medium | Isolation | Asked at: PhonePe, Flipkart, JP Morgan India

THE INTERVIEWER ASKS

> READ COMMITTED is the default in SQL Server. What does it prevent, and what does it still let through?

ANSWER IN 30 SECONDS

READ COMMITTED prevents dirty reads: you only see committed data. It still allows non-repeatable reads and phantom reads. In SQL Server it has two implementations: locking (the on-premises default) and row versioning, when READ_COMMITTED_SNAPSHOT is ON.

BUILD THE ANSWER STEP BY STEP

1. Locking READ COMMITTED: a reader takes a shared lock on each row or page and releases it as soon as it has read it, so a writer holding an exclusive lock blocks the reader.
2. READ COMMITTED SNAPSHOT (RCSI): each **statement** reads the last committed version as of its start, from the version store. Readers don't block writers and writers don't block readers.
3. Azure SQL Database enables RCSI by default; on-premises SQL Server does not.
4. Still possible: reading the same row twice in one transaction can return different values, and re-running a range query can return new rows.

EDGE CASES THAT IMPRESS

- Under RCSI the snapshot is per statement, not per transaction. Two SELECTs in one transaction can still disagree.
- Use `SELECT ... WITH (UPDLOCK)` when you read a row you intend to update, so another session cannot change it in between.
- Switching an existing system to RCSI can change behaviour of code that relied on blocking, such as some queue or counter patterns. Test before enabling.

AVOID SAYING

- "READ COMMITTED prevents all anomalies." It prevents one.
- "Readers never block in SQL Server." True only with RCSI or SNAPSHOT.

Likely follow-up: Give an example where a non-repeatable read causes a bug.

A procedure checks the wallet balance, does other work, then reads the balance again to calculate a fee. Another session changed the balance in between, so the fee is calculated on a different amount than the check approved.

Non-repeatable read in one transaction T-SQL

```
BEGIN TRAN;
SELECT Balance FROM dbo.Wallets WHERE WalletID = 101;    -- 5000
-- another session updates it to 8000 and commits
SELECT Balance FROM dbo.Wallets WHERE WalletID = 101;    -- 8000
COMMIT;

-- Is RCSI on?
SELECT name, is_read_committed_snapshot_on
FROM sys.databases WHERE name = DB_NAME();
```

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

45 REPEATABLE READ

Medium | Isolation | Asked at: Swiggy, Meesho, Walmart Labs

THE INTERVIEWER ASKS

> What does REPEATABLE READ add on top of READ COMMITTED, and how does SQL Server implement it?

ANSWER IN 30 SECONDS

REPEATABLE READ guarantees that rows you have read cannot be changed by others until your transaction ends, so re-reading them gives the same values. SQL Server implements it by holding shared locks until commit. It does **not** prevent phantoms: other sessions can still insert new rows that match your range.

BUILD THE ANSWER STEP BY STEP

1. Mechanism: shared locks on every row read are held to the end of the transaction instead of being released immediately.
2. Prevents dirty reads and non-repeatable reads.
3. Phantoms remain possible because SQL Server locks the rows that exist, not the gaps between them.
4. Compare engines: MySQL InnoDB uses gap locks at REPEATABLE READ, and PostgreSQL uses a snapshot, so both prevent most phantoms. SQL Server's REPEATABLE READ is weaker in that respect.

EDGE CASES THAT IMPRESS

- Holding shared locks longer causes more blocking: writers wait for every reader's transaction to finish.
- Classic **conversion deadlock**: two sessions read the same row (both hold S locks), then both try to update it. Each waits for the other's S lock. Read with UPDLOCK to avoid it.
- If you want consistency without blocking, SNAPSHOT isolation (Question 47) is usually the better tool.

AVOID SAYING

- "REPEATABLE READ prevents phantoms." Not in SQL Server.
- Using it without considering the extra blocking.

Likely follow-up: Why does MySQL default to REPEATABLE READ when SQL Server defaults to READ COMMITTED?

Largely historical: MySQL's statement-based replication needed stronger consistency, and InnoDB's MVCC makes REPEATABLE READ cheap for readers. SQL Server's locking implementation made READ COMMITTED the practical default.

Locks held until commit T-SQL

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRAN;
SELECT OrderID, Amount FROM dbo.Orders WHERE CustomerID = 7;
-- Other sessions cannot UPDATE these rows now,
-- but they CAN insert new orders for customer 7 (phantoms).

SELECT resource_type, request_mode, request_status
FROM sys.dm_tran_locks WHERE request_session_id = @@SPID;
COMMIT;
```

46

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

SERIALIZABLE

Hard | Isolation | Asked at: Goldman Sachs India, JP Morgan, Razorpay

THE INTERVIEWER ASKS

> What does SERIALIZABLE guarantee, and why don't we always use it?

ANSWER IN 30 SECONDS

SERIALIZABLE guarantees the result is the same as if transactions had run one after another. SQL Server achieves this with **key-range locks**, which lock not only the rows read but the ranges between them, so nobody can insert a phantom. The price is heavy blocking and more deadlocks under contention.

BUILD THE ANSWER STEP BY STEP

1. Strongest level in the SQL standard: no dirty, non-repeatable or phantom reads, and no write skew.
2. SQL Server places RangeS-S locks on the index keys covering the predicate. An insert into that range waits.
3. The `HOLDLOCK` table hint applies SERIALIZABLE semantics to a single table in a query, which is how MERGE upserts are made safe.
4. Use it for short, critical transactions, such as allocating the last unit of stock or posting to a ledger.

EDGE CASES THAT IMPRESS

- Key-range locks need a suitable index on the predicate column. Without one, SQL Server may lock far more, up to the whole table.
- PostgreSQL implements SERIALIZABLE optimistically (SSI) and aborts conflicting transactions; SQL Server blocks instead. Different failure mode, same guarantee.
- Oracle's SERIALIZABLE is actually snapshot isolation and does not prevent write skew.

AVOID SAYING

- "It's just slower." The real cost is blocking and deadlocks.
- "SERIALIZABLE means single-threaded." Concurrency continues; outcomes must match some serial order.

Likely follow-up: What is write skew? Give an example.

Rule: at least one doctor must be on call. Two on-call doctors each check that the other is on call, then each sets themselves off call. Both transactions were individually valid; together they break the rule. SERIALIZABLE prevents it by locking the range both read.

Allocate stock without phantoms or double-booking T-SQL

```
SET XACT_ABORT ON;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRAN;
    IF (SELECT COUNT(*) FROM dbo.Bookings
        WHERE SlotID = @SlotID) < @Capacity
        INSERT dbo.Bookings (SlotID, CustomerID) VALUES (@SlotID, @CustomerID);
    ELSE
        THROW 50001, 'Slot full', 1;
COMMIT;
```

47

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

Row Versioning: RCSI and SNAPSHOT Isolation

Hard | Transactions | Asked at: Zerodha, PhonePe, Flipkart, JP Morgan

THE INTERVIEWER ASKS

Explain multi-version concurrency control. How does SQL Server let readers see consistent data without blocking writers?

ANSWER IN 30 SECONDS

When row versioning is enabled, every update or delete first copies the previous committed version of the row into a **version store** and links it to the row through a 14-byte versioning tag. Readers follow that chain to the version that was committed when their statement (RCSI) or transaction (SNAPSHOT) began. Readers and writers therefore stop blocking each other.

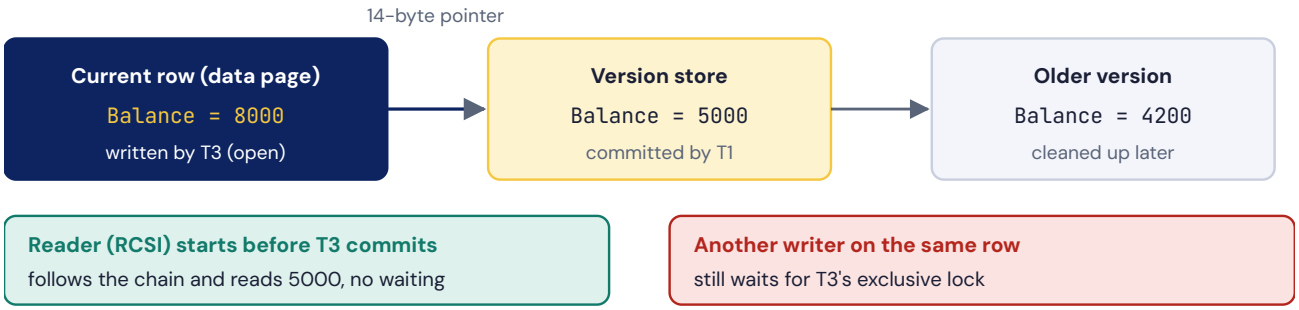


Figure 47. Row versioning: readers follow the chain to the last committed version

BUILD THE ANSWER STEP BY STEP

1. Two options: READ_COMMITTED_SNAPSHOT (statement-level consistency, transparent to code) and ALLOW_SNAPSHOT_ISOLATION, used with `SET TRANSACTION ISOLATION LEVEL SNAPSHOT` (transaction-level consistency).
2. The version store lives in tempdb. With Accelerated Database Recovery (SQL Server 2019+), versions are kept in a persistent version store inside the user database.
3. Writers still lock: two sessions updating the same row still block each other. Versioning only removes reader-writer blocking.
4. Under SNAPSHOT, if you update a row that another transaction changed after your snapshot began, SQL Server raises **error 3960** (update conflict) and the transaction must be retried.

EDGE CASES THAT IMPRESS

- A long-running SNAPSHOT transaction prevents old versions from being cleaned up, so tempdb (or the PVS) can grow sharply.
- Enabling versioning adds the 14-byte tag to modified rows, which can cause page splits on dense tables.
- Monitor with `sys.dm_tran_version_store_space_usage` and `sys.dm_tran_active_snapshot_database_transactions`.

AVOID SAYING

- "Versioning means no locks." Write locks remain.
- "RCSI and SNAPSHOT are the same." Statement-level vs transaction-level consistency, and only SNAPSHOT raises update conflicts.

Likely follow-up: What happens if the version store grows out of control?

tempdb fills, and SQL Server may stop generating versions, causing snapshot readers to fail. Find and end the oldest active snapshot transaction, then size tempdb for the workload.

Enable and use SNAPSHOT isolation T-SQL

```
ALTER DATABASE ShopSmartIndia SET ALLOW_SNAPSHOT_ISOLATION ON;

SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
BEGIN TRAN;
    SELECT SUM(Balance) FROM dbo.Wallets;    -- consistent as of BEGIN
    -- ... other reads see the same snapshot ...
COMMIT;

-- Oldest snapshot transactions holding versions
SELECT session_id, elapsed_time_seconds
FROM sys.dm_tran_active_snapshot_database_transactions
ORDER BY elapsed_time_seconds DESC;
```

48

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

Optimistic vs Pessimistic Locking

Medium | Transactions | Asked at: Razorpay, Swiggy, Meesho, TCS

THE INTERVIEWER ASKS

When would you lock a row up front versus check a version column at write time? What is the trade-off?

ANSWER IN 30 SECONDS

Pessimistic locking reads the row with `WITH (UPDLOCK)` so others wait until you commit; use it when conflicts are likely and transactions are short. Optimistic locking reads freely and checks at write time that nobody else changed the row, usually through a `rowversion` column; use it when conflicts are rare and you don't want to hold locks while a user thinks.

BUILD THE ANSWER STEP BY STEP

1. Pessimistic: `SELECT ... FROM dbo.Inventory WITH (UPDLOCK, ROWLOCK) WHERE ProductID = @p` inside the transaction, then UPDATE.
2. Optimistic: SQL Server's `rowversion` column changes automatically on every update. Read it, then `UPDATE ... WHERE ProductID = @p AND RowVer = @readVer`. If @@ROWCOUNT = 0, someone else changed it: retry or report a conflict.
3. Pessimistic fits stock decrements and wallet debits. Optimistic fits profile edits and admin screens.
4. For cross-row coordination that fits neither, SQL Server offers application locks through `sp_getapplock`.

EDGE CASES THAT IMPRESS

- `WITH (NOWAIT)` or `SET LOCK_TIMEOUT 0` fails immediately instead of waiting, useful in API hot paths.
- `WITH (READPAST)` skips locked rows, SQL Server's equivalent of SKIP LOCKED. It is ideal for job-queue tables.
- A rowversion value is unique within the database and always increasing, but it is not a date or time.

AVOID SAYING

- "Optimistic is always faster." Under heavy contention, retries thrash.
- "UPDLOCK blocks all readers." Plain readers still read; it blocks other UPDLOCK and exclusive requests.

Likely follow-up: How do you build a job-queue table so two workers never pick the same job?

Claim with one statement: `UPDATE TOP (1) q SET Status = 'RUNNING', Worker = @w OUTPUT inserted.JobID FROM dbo.JobQueue q WITH (ROWLOCK, READPAST, UPDLOCK) WHERE Status = 'READY'`. Locked rows are skipped, so workers never collide.

Optimistic update with rowversion T-SQL

```
ALTER TABLE dbo.Customers ADD RowVer rowversion;

-- Read
SELECT CustomerID, City, RowVer FROM dbo.Customers WHERE CustomerID = 7;

-- Write only if unchanged since the read
UPDATE dbo.Customers
   SET City = @NewCity
 WHERE CustomerID = 7
   AND RowVer = @RowVerFromRead;

IF @@ROWCOUNT = 0
    THROW 50002, 'Row was changed by another user. Reload and retry.', 1;
```

49

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

Deadlocks: Detection and Retry

Medium | Transactions | Asked at: Razorpay, Cred, Walmart Labs, Infosys

THE INTERVIEWER ASKS

What is a deadlock, how does SQL Server handle it, and what should the application do?

ANSWER IN 30 SECONDS

A deadlock is a cycle: transaction A holds a lock B needs, while B holds a lock A needs. SQL Server's lock monitor detects the cycle, chooses a victim, rolls it back and returns **error 1205**. The application should catch 1205 and retry the whole transaction a limited number of times.

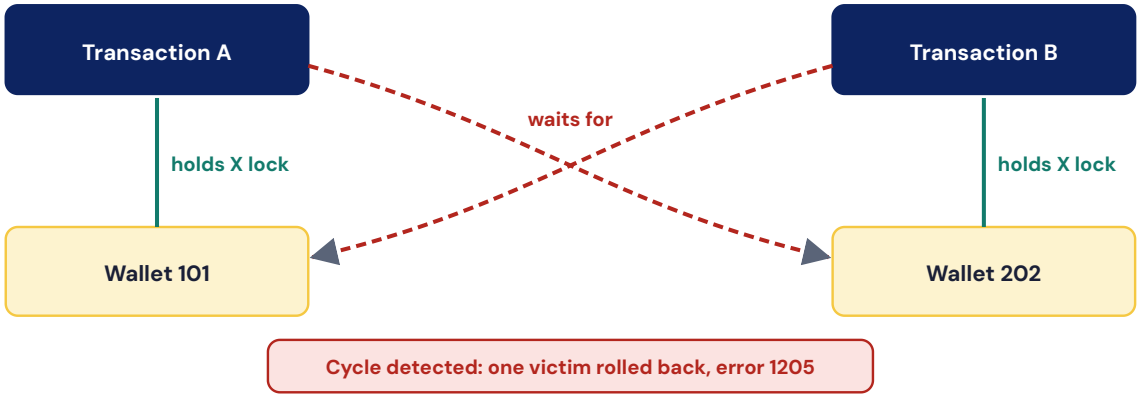


Figure 49. A deadlock is a cycle of waits

BUILD THE ANSWER STEP BY STEP

1. Classic case: A updates wallet 101 then wants 202; B updates wallet 202 then wants 101.
2. Detection: a background lock monitor checks for cycles every few seconds, and more often once deadlocks are found.
3. Victim choice: by default the transaction cheapest to roll back. `SET DEADLOCK_PRIORITY LOW` volunteers a session as the victim.
4. Prevention: access objects in a consistent order, keep transactions short, add indexes so fewer rows are locked, and consider RCSI to remove reader-writer deadlocks.

EDGE CASES THAT IMPRESS

- The built-in **system_health** Extended Events session records every deadlock graph (`xml_deadlock_report`), so you can investigate after the fact.
- A lock timeout (error 1222, after `SET LOCK_TIMEOUT`) is not a deadlock; it is a wait that gave up.
- Deadlocks across different databases or services are not detected by one server. They need timeouts and retries in the application.

AVOID SAYING

- "Deadlocks mean the schema is broken." Busy systems can deadlock with good design.
- "Just retry forever." Use bounded retries with a short back-off.

Likely follow-up: How can you reduce deadlocks without changing application logic?

Add or adjust indexes so queries lock fewer rows, enable READ_COMMITTED_SNAPSHOT to remove reader-writer deadlocks, and shorten transactions by moving non-database work outside them.

Retry on deadlock (error 1205) T-SQL

```
DECLARE @try int = 1;
WHILE @try ⩽ 3
BEGIN
    BEGIN TRY
        SET XACT_ABORT ON;
        BEGIN TRAN;
            -- Lock in a consistent order: lower WalletID first
            UPDATE dbo.Wallets SET Balance = Balance - 500 WHERE WalletID = 101;
            UPDATE dbo.Wallets SET Balance = Balance + 500 WHERE WalletID = 202;
        COMMIT;
        BREAK;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        IF ERROR_NUMBER() = 1205 AND @try < 3
        BEGIN
            SET @try += 1;
            WAITFOR DELAY '00:00:00.200';
        END
        ELSE THROW;
    END CATCH
END;
```

50

PART 6 · TRANSACTIONS, ISOLATION & CONCURRENCY

Lock Escalation

Hard | Transactions | Asked at: AmEx India, Goldman Sachs, Wipro

THE INTERVIEWER ASKS

> Explain lock escalation in SQL Server. When does a row lock become a table lock?

ANSWER IN 30 SECONDS

Lock escalation is SQL Server converting many fine-grained row or page locks into a single table lock to save lock memory. It is attempted when one statement acquires roughly **5,000 locks** on a single table or index, or when lock memory pressure is high. Once escalated, other sessions cannot modify any row in that table until the transaction ends.

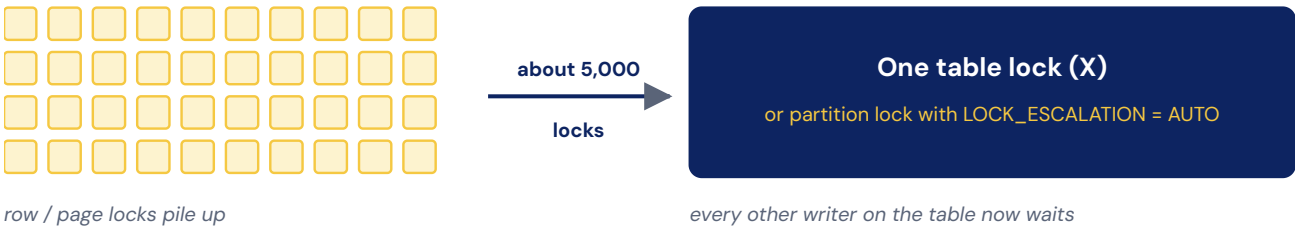


Figure 50. Lock escalation from many fine-grained locks to one table lock

BUILD THE ANSWER STEP BY STEP

1. Why: every lock consumes memory in the lock manager. Millions of row locks are expensive.
2. Path: row or page locks escalate directly to a table lock (or a partition lock). SQL Server does not escalate row locks to page locks.
3. Effect: a large UPDATE that escalates to an exclusive table lock blocks every other writer, and readers too under locking READ COMMITTED.

4 Avoid it by batching large changes into chunks below the threshold, keeping transactions short, and using `LOCK_ESCALATION = AUTO` on partitioned tables so escalation stops at the partition.

EDGE CASES THAT IMPRESS

- If another session holds an incompatible lock on the table, escalation fails, and SQL Server keeps the fine-grained locks and tries again later.
- `ALTER TABLE ... SET (LOCK_ESCALATION = DISABLE)` prevents escalation in most cases, at the cost of more lock memory. Use it rarely.
- Oracle and PostgreSQL do not escalate row locks in this way, so this question is mostly about SQL Server and Db2.

AVOID SAYING

- "All databases escalate locks."
- "Escalation is always bad." For an off-hours batch, a table lock can be intentional and faster.

Likely follow-up: How do you delete 10 million rows without causing lock escalation?

Delete in batches with `DELETE TOP (4000)` in a loop until @@ROWCOUNT is 0, each batch in its own short transaction, optionally pausing between batches. For whole months on a partitioned table, TRUNCATE or SWITCH the partition instead.

Batched delete below the escalation threshold T-SQL

```
DECLARE @rows int = 1;
WHILE @rows > 0
BEGIN
    DELETE TOP (4000) FROM dbo.Orders
    WHERE Status = 'EXPIRED'
      AND OrderDate < DATEADD(YEAR, -3, GETDATE());

    SET @rows = @@ROWCOUNT;
    WAITFOR DELAY '00:00:00.100';    -- give other sessions room
END;
```

Before you walk into the interview

THE TEN TRAPS THAT DECIDE MOST SQL ROUNDS

1. A right-table filter in WHERE turns a LEFT JOIN into an INNER JOIN. (Q01)
2. One NULL in a NOT IN subquery returns zero rows. Prefer NOT EXISTS. (Q07)
3. Joins multiply rows; EXISTS does not. (Q08)
4. ROW_NUMBER drops ties; DENSE_RANK keeps them. (Q12)
5. The default window frame is RANGE, not ROWS. Write the frame. (Q17)
6. LAST_VALUE needs an explicit frame to the end of the partition. (Q18)
7. Window functions cannot go in WHERE, and SQL Server has no QUALIFY. (Q21)
8. Functions on a column and implicit conversions block index seeks. (Q37)
9. SET XACT_ABORT ON, or a failed statement can leave a transaction half-done. (Q41)
10. SQL Server's REPEATABLE READ still allows phantoms. (Q45)

A FINAL WORD

Interviewers rarely reject someone for forgetting syntax. They reject people who cannot explain why. If you can draw the diagram and say the edge case for each question in this playbook, you are ready. Practise on real data, say your answers out loud, and walk in calm. All the best.